Transformer-Based Time-Series Forecasting of the Kuramoto-Sivashinsky Chaotic System

By

Zhang YUE

Supervisor: Vikrant GUPTA

Guangdong Technion – Israel Institute of Technology

Mechanical Engineering and Robotics

April 2026, Guangdong

Table of Contents

Abstract 3

Overall Workflow KS equation Prediction Project 4

Chapter 1. Data generation 5

1.1 Continuous form and spatial interpretation 5

1.2 Discretization and numerical representation 6

Chapter 2. Training sample construction 8

2.1 Initial condition and trajectory 8

2.2 Time marching and raw trajectory generation 12

2.3 Snapshot, sample and sliding window 13

2.4 Dataset size, train split and batch size 14

2.5 Epoch and Training Iteration 16

Chapter 3. Model Architecture 17

3.1 Model Input and Output 17

3.2 Self-Attention Mechanism 20

3.2.1 Input Projection 20

3.2.2 Positional Encoding 22

3.2.3 Query, Key, and Value 27

3.2.4 Attention Scores, Normalization, and SoftMax 31

3.2.5 Attention Output 34

3.2.6 Multi-Head Self-Attention 37

3.2.7 Final Output Projection 41

3.3 Easy Attention Mechanism 46

3.3.1 Core Idea of Easy Attention 46

3.3.2 Why Easy Attention Is Simpler 50

Chapter 4. Training Strategy and Optimization 52

4.1 Learning Objective and Loss Function 52

4.2 Gradient Descent: Why Parameters Move 56

4.3 Backpropagation Through the Model 59

4.4 How the Matrices Are Learned 62

4.5 Adam Optimizer (Adaptive Moment Estimation) 65

4.6 Learning Rate Decay 70

4.7 Gradient Clipping 72

4.8 Validation Monitoring and Best Model Selection 74

Chapter 5. Prediction and Evaluation Using NRMSE 76

5.1 Prediction Procedure: Short-Term and Autoregressive Forecasting 76

5.2 NRMSE Calculation and Interpretation 78

Chapter 6. Downsampling Results and Discussion 83

Abstract

The Kuramoto–Sivashinsky (KS) equation is a nonlinear partial differential equation that describes complex spatiotemporal behavior and is well known for its chaotic nature. Its standard form is

$$u_t + u\,u_x + u_{xx} + u_{xxxx} = 0,$$

where $u(x,t)$ represents the system state evolving over space and time. Due to the combination of nonlinear and higher-order terms, the system is highly sensitive to initial conditions. Small differences can grow rapidly over time, which makes long-term prediction inherently difficult.

In this project, instead of repeatedly solving the equation using numerical methods such as ETDRK4, a data-driven approach is explored. A transformer-based model is trained on simulated data to learn how the system evolves over time. Given a sequence of past states, the model predicts future states, and this prediction can be extended step by step to generate longer trajectories. In this way, the model acts as an approximation of the underlying dynamics, allowing fast prediction without explicitly solving the equation each time.

From a deep learning perspective, this task can be understood as learning a mapping from a short time window of the system to its immediate future. The attention mechanism helps the model identify which past time steps are more relevant for prediction, while the feedforward layers provide the necessary nonlinear transformation. Compared with traditional sequential models, the transformer processes the entire input window at once, which makes it more flexible in capturing temporal patterns. In this project, a simplified attention mechanism is used to reduce complexity while maintaining the ability to extract useful temporal information.

The prediction performance is evaluated using the normalized root mean square error (NRMSE), which measures the prediction error relative to the scale of the true signal. This is particularly important for chaotic systems, where the amplitude can vary over time. The revised experiments and their evaluation convention appear in Chapter 6; Chapters 1–5 retain the original prototype and its worked examples.

More broadly, this project reflects a shift from solving equations to learning system behavior from data. In many mechanical and robotic systems, the exact governing equations may be difficult to obtain or too complex for real-time computation. A learned model that can approximate system evolution offers a practical alternative, especially for tasks such as prediction, control, and system identification. Therefore, this work not only explores the prediction of a chaotic system, but also provides insight into how data-driven methods can be applied in engineering contexts.

Overall Workflow of the Kuramoto–Sivashinsky (KS) equation Prediction Project

(1) Data generation.
At this stage, the neural network is not involved yet. We first specify the parameters of the KS equation, such as the spatial domain length L, the number of spatial grid points N, the time step dt, and the total simulation time. Then, multiple different initial conditions are generated, for example many different $u(x,0)$. Each initial condition is one spatial curve at $t=0$. Starting from each initial condition, the KS equation is solved forward in time using a numerical method such as ETDRK4. In this way, each initial condition produces one full trajectory, which can be viewed as a spatiotemporal surface $u(x,t)$. After these long trajectories are obtained, the early transient part may be removed, spatial (x axis) downsampling may be applied if needed, and the results are saved as the dataset. In other words, this stage is not prediction; it is the process of generating the ground-truth data in advance by solving the PDE numerically.

(2) Training sample construction.
This stage is still not model training. It is the preparation of training materials. Once the long trajectories are available, they are cut into many smaller samples. In the original prototype, the input length is seq_len = 64 and the prediction length is pred_len = 3. Therefore, one sample is constructed as follows:

$$[u^1, u^2, \dots, u^{64}] \rightarrow [u^{65}, u^{66}, u^{67}]$$

where each u^k is a spatial vector at a fixed time step. Then the window keeps sliding forward:

$$[u^2, \dots, u^{65}] \rightarrow [u^{66}, u^{67}, u^{68}]$$
$$[u^3, \dots, u^{66}] \rightarrow [u^{67}, u^{68}, u^{69}]$$

and so on. So the essence of this stage is to cut each long trajectory into many small input-target windows of the form “past 64 steps →next 3 steps.” This is also the stage where the notion of a training sample first appears.

(3) Model training.
This is the stage where the model is actually trained. During training, a batch of samples is taken from the whole sample set. For example, if batch_size = 32, then 32 samples are fed into the model at one time. If the spatial dimension is also 64, then the input tensor has the approximate shape: $(32,64,64)$,
which means 32 samples, 64 time steps in each sample, and 64 spatial points at each

time step. The model outputs its own prediction for the next 3 time steps. The prediction is then compared with the true target, and a loss function such as MSE is computed:

$$\text{MSE} = \mathcal{L} = \frac{1}{n}\sum(\hat{y} - y)^2.$$

This error is backpropagated to update the model parameters. By repeating this process many times, the model gradually learns how to use the past 64 steps to predict the next 3 steps more accurately. So in this stage, training performance is mainly reflected by whether the loss decreases.

(4) Prediction and evaluation.
This stage is different from training. During training, the model learns from true past inputs and true future targets. During prediction, the trained model is used to make actual forecasts. For short-term prediction, the model is given a true input window, for example

$$[u^1, \dots, u^{64}],$$

and directly predicts

$$[\hat{u}^{65}, \hat{u}^{66}, \hat{u}^{67}].$$

These predictions are then compared with the true numerical solution u^{65}, u^{66}, u^{67}. For longer-horizon prediction, an autoregressive strategy can be used: the model first predicts the next 3 steps, then these newly predicted states are fed back into the input, and the model continues to roll forward in time. This is the true autoregressive rollout. Finally, the model performance is evaluated mainly by the NRMSE. This is more convincing than training loss alone, because a low training loss does not necessarily mean good long-term prediction performance. In this project, the main concerns are whether the short-term prediction is accurate, whether the long-term rollout stays stable, and how the NRMSE behaves during prediction.

Chapter 1. Data generation

1.1 Continuous form and spatial interpretation

When we study the Kuramoto–Sivashinsky (KS) equation, we are still dealing with a partial differential equation with two independent variables: space x and time t. So the unknown is written as $u(x,t)$. $x_1, x_2, x_3, \dots$ are only sampled spatial points obtained after discretizing the continuous spatial variable x. In continuous

mathematics, x can take infinitely many values in a spatial interval. But a computer cannot store infinitely many points, so we replace the continuous interval with a finite number of grid points. For example, if the spatial interval is $[0, L)$, then we may divide it into N evenly spaced points. In this case, N is the number of spatial grid points.

At any fixed time $t = t_0$, the function $u(x, t_0)$ depends only on x. That means it is a function of space, so if we draw x on the horizontal axis and u on the vertical axis, we get one spatial curve. In a computer, that curve is represented by listing its values at the grid points.

For example, if the solution at time t_0 is sampled at five spatial points and the values are

$$u(x_1, t_0) = 1.2, u(x_2, t_0) = 0.4, u(x_3, t_0) = -0.3, u(x_4, t_0) = 0.1, u(x_5, t_0) = 0.9,$$

then the whole spatial state at that time can be written as a vector

$$u(t_0) = [1.2,\ 0.4,\ -0.3,\ 0.1,\ 0.9]$$

If we do this for many time instants, then we obtain a time sequence of vectors. That is exactly the kind of data used for machine learning prediction.

1.2 Discretization and numerical representation

For the KS system in this project, the spatial interval length is chosen as $L = 22$. This means we study the system on a finite spatial domain of length 22. The code uses $N = 64$ spatial points, so the original spatial grid is quite fine. The reason for choosing $N = 64$ is practical rather than fundamental: it is large enough to describe the spatial structure well, and it is also a power of 2, which is convenient for numerical methods such as Fourier-based methods and Fast Fourier Transform (FFT). The spatial points are usually generated as

$$x \in [0, L),$$

so the first point is $x_0 = 0$, and the last point is just before L. If the code uses something like np.linspace(0, L, N, endpoint=False), then $x = 0$ is included but $x = L$ is not included. This is common for periodic problems, because $x = 0$ and $x = L$ represent the same physical location, so including both would be redundant.

The spatial step size is determined by

$$dx = \frac{L}{N}.$$

With $L = 22$ and $N = 64$, we get

$$dx = \frac{22}{64} \approx 0.34375.$$

This means neighboring spatial sample points are about 0.34 units apart. A smaller Δx means finer spatial resolution.

Time is also discretized. Instead of computing the solution at every possible real value of time, we compute it at equally spaced time points. In this project, the time step is chosen as $dt = 0.1$. So the time values look like:

$$t_0 = 0, t_1 = 0.1, t_2 = 0.2, t_3 = 0.3, \ldots$$

If the total simulation time is chosen as $t_{\text{total}} = 20000$s, then the time interval is $t \in [0,20000]$. This means the computer generates solution snapshots from time 0 up to time 20000, with spacing 0.1 between neighboring snapshots. At each of these times, there is one spatial curve, and after discretization, one vector.

A very important idea is that the computer is not working with exact continuous function $u(x,t)$ everywhere. Instead, it works with an approximation on a discrete grid. That means the prediction task is also discrete. The model does not directly predict the exact continuous solution for every possible x and every possible t. Instead, it predicts the values of the solution on selected spatial points and selected time steps. Therefore, this approach has limitations. First, the spatial domain is finite, so we only study behavior on a limited interval rather than the whole infinite line. Second, space is discretized into N points, so the model only sees sampled values rather than the full continuous curve. Third, time is also discretized, so the model only predicts the solution at specific time steps rather than at every possible instant. Because of this, accuracy depends strongly on the grid resolution. If dx is too large, fine spatial details may be lost. If dt is too large, the time evolution may be too coarse. So in general, to obtain a more accurate approximation to the continuous PDE, we want sufficiently small spatial spacing and sufficiently small time spacing.

In short, the KS equation is still a two-variable PDE in x and t. The symbols $x_1, x_2, x_3, \ldots$ are not extra variables; they are just sampled spatial locations after discretization. At a fixed time, $u(x,t)$ becomes a function of space only, so it can be viewed as one spatial curve. In the computer, that curve is stored as a vector of sampled values. Repeating this over many time steps gives a sequence of vectors, which is the data used for prediction. In this project, the main numerical choices are:

spatial length $L = 22$, number of spatial points $N = 64$, spatial step size $\Delta x =$

$L/N = 22/64$, time step $dt = 0.1$, and time interval approximately $t \in [0,20000]$. These choices make the continuous PDE computable, but they also introduce approximation errors because the continuous system is replaced by a discrete one.

Chapter 2. Training sample construction

2.1 Initial condition and trajectory

For the KS equation, the unknown is a function u(x,t). At the beginning of a simulation, we must specify the solution at time t=0. This starting curve is called the initial condition: u(x,0). Because t=0 is fixed, this is a function of space only. So it is one spatial curve. After discretization, that curve becomes a vector of values on the spatial grid points. For example, if the spatial grid has five points, one initial condition may look like:

$$u(x,0) = [1.2, 0.4, -0.3, 0.1, 0.9]$$

Time is fixed at t=0 and the space locations $x_1, x_2, \ldots$are fixed grid points. The values of the curve at those points are listed in a vector. So an initial condition is simply: one starting spatial profile at time t=0. In the code, the number of initial conditions is controlled by:

```
ics = generate_initial_conditions(config.L, config.N, n_ics=100)
```

The key number is n_ics=100, which means the code generates 100 different initial conditions. These are 100 different starting curves at t=0 (does not mean 100 training epochs). It means 100 different starting states for the PDE solver. If we only used one initial condition, we would only get one trajectory. That would make the dataset too limited. By using 100 different initial conditions, we get 100 different evolutions of the KS system. Across the 100 trajectories: the spatial grid and time step are the same. This is useful because the KS system is chaotic and sensitive to initial conditions. That means: if the starting curve changes, the later evolution can also change a lot. So the purpose of multiple initial conditions is to create more data and expose the model to different KS behaviors, which can help the model learn more general dynamics.

A KS “trajectory” here does not mean a simple 2D curve like in basic mechanics. Instead, one trajectory is better understood as a spatiotemporal surface**.** Because the solution depends on both: space x and time t. So one full numerical solution is u(x,t) with horizontal axis = x, vertical axis = t, height = u(x,t), then the result looks like a surface. In practice, the computer stores this surface as many discrete points: $(x_i, t_k, u(x_i, t_k))$.

How is each trajectory generated? After generating an initial condition, the code solves the KS equation forward in time using a numerical method called **ETDRK4(Exponential Time Differencing Fourth-Order Runge-Kutta)**.

In this project, the equation is written as

$$u_t + uu_x + u_{xx} + u_{xxxx} = 0,$$

or equivalently,

$$u_t = -uu_x - u_{xx} - u_{xxxx}.$$

This equation contains both a nonlinear term, $-uu_x$, and higher-order derivative terms, especially the fourth-order derivative $-u_{xxxx}$. These higher-order derivatives make the system numerically stiff, which means that a standard explicit time-stepping method may require an extremely small time step for stability. For this reason, a more suitable method is needed for data generation.

A standard Runge–Kutta method approximates the solution of an ordinary differential equation by evaluating the slope several times within one time step and combining these slope estimates to achieve higher accuracy. This idea works well for many non-stiff problems. However, for the KS equation, applying an ordinary RK scheme directly to the full right-hand side is inefficient, because the linear derivative terms can evolve much more rapidly than the nonlinear term. The key idea of ETDRK4 is therefore to treat the linear and nonlinear parts differently.

To make this possible, the KS equation is first rewritten in the form

$$u_t = Lu + N(u),$$

where L denotes the linear operator and $N(u)$denotes the nonlinear part. For the KS equation in this project, one may identify

$$Lu = -u_{xx} - u_{xxxx}, \qquad N(u) = -uu_x.$$

Here, L is not an ordinary scalar constant. It is a linear operator acting on the function $u(x,t)$. In the continuous equation, this operator contains the second- and fourth-order spatial derivatives. After discretization, it becomes a matrix acting on the discrete solution vector. Therefore, the higher-order derivatives have not disappeared. They are absorbed into the operator L, which compactly represents the linear part of the PDE.

The next step is to handle the spatial derivatives efficiently. This is where the Fourier spectral method is used. The basic idea is to represent the spatial solution $u(x,t)$as a sum of Fourier modes. For a periodic problem, this is especially natural, because a periodic spatial function can be expanded in terms of complex exponentials or, equivalently, sine and cosine modes. If $u(x,t)$is transformed into Fourier space, then differentiation with respect to $x$becomes multiplication by powers of ik, where $k$is the wave number. In particular,

$$\frac{\partial}{\partial x} \longrightarrow ik, \qquad \frac{\partial^2}{\partial x^2} \longrightarrow (ik)^2, \qquad \frac{\partial^4}{\partial x^4} \longrightarrow (ik)^4.$$

This is the main advantage of the Fourier spectral method. In physical space, computing high-order derivatives directly can be cumbersome and inaccurate. In Fourier space, the same derivatives become simple multiplications. As a result, the linear operator L becomes diagonal, or at least very easy to evaluate mode by mode. This is why the Fourier spectral method is especially effective for the KS equation.

In practice, the transformation between physical space and Fourier space is carried out using the Fast Fourier Transform (FFT). The FFT is not a different mathematical method from the Fourier transform itself; it is simply a fast algorithm for computing the discrete Fourier transform and its inverse. In this project, FFT is used repeatedly because the solution must move between two representations. The derivative terms are easiest to compute in Fourier space, while the nonlinear term $uu_x$is most naturally formed in physical space. Therefore, the numerical procedure alternates between these two domains: transform to Fourier space, evaluate derivatives, transform back to physical space, evaluate nonlinear products, and then continue the time-stepping process.

Once the spatial discretization is completed, the PDE becomes a system of ordinary differential equations for the Fourier coefficients, which can be written abstractly as

$$\frac{dU}{dt} = LU + N(U),$$

where $U(t)$ now represents the discrete state of the system at time t. At this point, the reason for the exponential term in ETDRK4 becomes clear. If we temporarily ignore the nonlinear term and only consider the linear equation

$$\frac{dU}{dt} = LU,$$

then its exact solution is

$$U(t + \Delta t) = e^{L\Delta t}U(t).$$

This exponential expression is not introduced artificially. It comes directly from solving the linear differential equation. In the scalar case, if $y_t = \lambda y$, then the exact solution is $y(t) = e^{\lambda t}y(0)$. The operator exponential $e^{L\Delta t}$is the direct generalization of this idea to the matrix or operator setting. This is the origin of the exponential term in ETDRK4.

The purpose of ETDRK4 is to use this exact linear propagation while still retaining a fourth-order approximation for the nonlinear part. To derive the method, one starts from

$$\frac{dU}{dt} = LU + N(U)$$

and introduces an integrating factor. Multiplying by e^{-Lt}gives a transformed equation in which the linear term is removed from the derivative. After integrating over one time step, one obtains the exact identity

$$U_{n+1} = e^{L\Delta t}U_n + \int_0^{\Delta t} e^{L(\Delta t-\tau)}\, N(U(t_n + \tau))\, d\tau.$$

This formula is the foundation of exponential time differencing. The first term represents the exact contribution of the linear part over one time step, while the integral term represents the accumulated contribution of the nonlinear part. The idea of ETDRK4 is then to approximate this nonlinear integral with fourth-order accuracy, in a way analogous to a Runge–Kutta method. In other words, the linear stiff part is treated analytically, and the nonlinear part is sampled several times within the time step to produce a fourth-order correction.

In implementation, one usually precomputes quantities such as

$$E = e^{L\Delta t}, E_2 = e^{L\Delta t/2},$$

together with several coefficient functions related to $L\Delta t$. These coefficients are then used to combine nonlinear evaluations at intermediate stages. Although the full ETDRK4 update formula is more complicated than the standard RK4 formula, its structure is conceptually simple: it is still a four-stage high-order method, but the linear part is handled through exact exponential propagation rather than through ordinary slope approximation.

For the KS equation, this combination is particularly suitable. The fourth-order derivative term, which is responsible for stiffness, is absorbed into the linear operator $L$and treated through the exponential propagator. The nonlinear transport term uu_x, which is essential for the chaotic dynamics of the system, is retained in the nonlinear correction. In this way, ETDRK4 achieves both stability and accuracy. It is far better suited to the KS equation than a standard explicit method that treats every term in exactly the same way.

So for each initial condition, start from u(x,0), use ETDRK4 to move forward in time to obtain the full evolution u(x,t).

100 initial conditions ⇒ 100 full trajectories ⇒ 100 spatiotemporal surfaces

2.2 Time marching and raw trajectory generation

Raw trajectory generation means numerically marching each initial condition forward in time to obtain a long KS trajectory before any sample cutting is performed.

The total simulation time is $t_{\text{total}} = 20000$
which means that each trajectory is simulated from $t = 0$ up to $t = 20000$.
Therefore, the total number of time steps in one full trajectory is

$$\frac{20000}{0.1} = 200000$$

This is where the number 200000 comes from. So each trajectory contains 200000 saved time steps. The code also removes the early transient part of the trajectory. This means the simulation starts at $t = 0$, but the very early part is considered a kind of

"warm-up" stage, strongly affected by the startup behavior, so it is removed before training. If the code removes the first 1000 units of time, then the number of removed time steps is

$$\frac{1000}{0.1} = 10000$$

So from the original 200000 time steps, the remaining number is

$$200000 - 10000 = 190000$$

This means that after removing the transient, each trajectory keeps 190000 time steps. Since there are 100 trajectories, the total number of saved time snapshots is

$$100 \times 190000 = 19{,}000{,}000$$

2.3 Snapshot, sample and sliding window

A snapshot is one fixed-time spatial state. That means choose one time t_k and take the full spatial curve at that time. After discretization, one snapshot is one vector: u^k=[u(x_1, t_k),u(x_2, t_k),…, u(x_N, t_k)]
one trajectory = many snapshots over time, one snapshot = one spatial vector at one fixed time. So one trajectory is a full spatiotemporal dataset produced from one initial condition. This is why it is reasonable to think of each trajectory as a surface. Each trajectory is generated independently. For each initial condition, the time starts from t=0 and then moves forward 0, dt, 2dt, 3dt … This does not mean the model always predicts starting from t=0. It only means that each numerically generated trajectory has a starting time. In other words, if we choose one time $t = t_k$, then the full spatial curve at that time is one snapshot. After discretization, one snapshot is one vector

$$u^k = [u(x_1, t_k), u(x_2, t_k), \dots, u(x_N, t_k)]$$

So one trajectory is made of many snapshots over time, and each snapshot is one spatial vector at one fixed time.The model does not receive one full trajectory at once. Instead, it receives small windows cut from long trajectories. Suppose the input length is seq_len = 64 and the prediction length is pred_len = 3. Then one training sample looks like this:

Input: $[u^1, u^2, \dots, u^{64}]$
Output: $[u^{65}, u^{66}, u^{67}]$

Here, each u^k is one snapshot, and each snapshot is one full spatial vector. So the input is a sequence of 64 spatial snapshots, and the output is the next 3 spatial snapshots. In plain language, the model is given 64 past states and asked to predict the next 3 states. These windows are not taken only from the beginning of the trajectory. Instead, the code uses a sliding window. That means the first sample may use steps 1–64 to predict 65–67, the second sample may use 2–65 to predict 66–68, the third sample may use 3–66 to predict 67–69, and so on. So the model is trained on many overlapping windows cut from the long trajectory. This is why the dataset can become very large.

2.4 Dataset size, train split and batch size

Dataset size refers to how many snapshots and how many sliding-window samples are available after trajectory generation and preprocessing. This also explains why we subtract 64 when counting samples. If a trajectory had 100 time steps and the input length were 64, then the possible input windows would be 1–64, 2–65, 3–66, all the way up to 37–100. So the number of possible windows would not be 100, but $100 - 64 + 1 = 37$ .If we also need 3 future steps as output, then the number becomes $100 - 64 - 3 + 1 = 34$.
So subtracting 64 is not mysterious at all. It simply means that one training sample must reserve 64 time steps for the input window.

Train split means dividing the available data into a training part and a testing/validation part according to a predefined ratio. The parameter train_ratio is also straightforward. It is just the fraction of data used for training. For example, $train_ratio = 0.8$ means that 80% of the available data is used for training and 20% is used for testing or validation.

If the total number of saved time snapshots is $19{,}000{,}000$, then the training part is $19{,}000{,}000 \times 0.8 = 15{,}200{,}000$
So, there are 15,200,000 time snapshots available for constructing training samples. If the training snapshots are 15,200,000, and seq_len = 64, and pred_len = 3, then the number of training samples is $15{,}200{,}000 - 64 - 3 + 1 = 15{,}199{,}934$
So the model is not trained on only a few samples. It is trained on a very large number of sliding windows.

Another important quantity is batch_size. The batch size determines how many training samples are used in one parameter update. It does not mean how many full trajectories are fed into the model at once.

In this project, one sample is a sliding window consisting of 64 past time steps as input and the next 3 time steps as the target. If the spatial dimension is 64, then one input sample has the shape

$$X \in \mathbb{R}^{64\times 64},$$

and one target sample has the shape

$$Y \in \mathbb{R}^{3\times 64}.$$

If the batch size is 32, then one training batch contains 32 such samples, and the input tensor has the form

$$X_{\text{batch}} \in \mathbb{R}^{32\times 64\times 64}.$$

These samples may come from different trajectories, or several of them may come from different positions of the same trajectory. Therefore, batch size mainly controls how many window samples are processed together in one training step. One batch loss is computed, and the model parameters are updated once based on that batch.

The choice of batch size is important because it directly affects gradient stability, computational efficiency, memory usage, and update frequency.

If the batch size is too small, such as 1 or only a few samples, the gradient is computed from very limited data and may fluctuate strongly from one update to the next. In that case, parameter updates become noisy, the training curve may oscillate more, and convergence may become slower or less stable. Although small batches allow more frequent updates, each update is based on less reliable information.

If the batch size is too large, the gradient becomes more stable because it is averaged over many samples, but the cost of each update becomes much higher. Large batches require more GPU memory and reduce the number of parameter updates per epoch. As a result, training may become less flexible and slower in terms of optimization progress, even if each gradient estimate is smoother.

Another practical reason for using mini batches is GPU parallelism. Modern GPUs are designed to process many samples simultaneously. Therefore, feeding 32 samples together is usually much more efficient than feeding them one by one. In this sense, mini-batch training not only improves optimization behavior but also makes better use of the hardware.

Therefore, batch size provides a practical trade-off. A very small batch leads to noisy gradients and unstable updates, while a very large batch increases memory cost and

reduces update frequency. In this project, a batch size of 32 is used as a moderate setting that balances gradient stability, GPU efficiency, and training speed.

2.5 Epoch and Training Iteration

In neural network training, one epoch does not correspond to a single parameter update. Instead, one epoch means that the entire training dataset is processed once.

In this project, the total number of training samples is

$$N_{\text{train}} = 15{,}199{,}934.$$

Training is performed using mini batches with batch size

$$B = 32.$$

Each batch contains 32 samples. For each batch, the model computes predictions, evaluates the loss against the ground truth, performs backpropagation, and updates the parameters once.

Therefore, the number of parameter updates (number of batches) in one epoch is

$$\left\lceil \frac{15{,}199{,}934}{32} \right\rceil = 474{,}998.$$

This means that one epoch consists of 474,998 update steps, during which all training samples are used exactly once (without repetition within the same epoch).

One epoch corresponds to traversing all sliding-window samples extracted from the trajectories generated by the 100 initial conditions.

Training is repeated for multiple epochs. In this project, the total number of epochs is

$$E = 500.$$

Thus, the total number of parameter updates during training is

$$500 \times 474{,}998 = 237{,}499{,}000.$$

The purpose of using multiple epochs is to repeatedly expose the model to the same dataset so that the parameters can be gradually refined. During each epoch, the dataset is shuffled, ensuring that the order of samples changes across epochs while each sample is still used exactly once per epoch.

train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)

During training, the loop

for x, y in train_loader:

iterates over all batches exactly once.

We do not shuffle time steps. We first construct sliding-window samples (64→3) and then shuffle these samples. Mini batches are formed by grouping multiple samples (batch size = 32).We shuffle samples, not batches. Batches are formed sequentially after shuffling.

Training for multiple epochs is necessary because one full pass over the dataset is usually not sufficient for the model to learn a good parameter configuration. During the first epoch, the parameters are still close to their initial random values, so the model can only make rough corrections. By repeating the training process over many epochs, the model revisits the same samples with gradually improved parameters, which allows the input-output mapping to be refined step by step. If only one epoch were used, the model would typically remain undertrained, with relatively high loss and unsatisfactory prediction accuracy.

Chapter 3. Model Architecture

3.1 Model Input and Output

At the model stage, the main question is no longer how the dataset is generated, but what the model actually receives as input and what it is expected to produce as output.

Training is performed in the following way: the model is given the first 64 time steps and asked to predict the next 3 time steps. The correct values of these 3 future steps are actually known in advance, but they are not shown to the model as input. They are only used after the prediction is made, in order to compare the model output with the true answer. The next 3 true time steps are used as targets for loss computation, not as part of the model input. In other words, one training sample is constructed from 67

consecutive time steps, but only the first 64 time steps are fed into the model. This is the basic idea of supervised learning in this project. In our model, self-attention is applied only to the observed 64-step input window. Since all these time steps are already known historical states, there is no risk of leaking future target information, and therefore no causal mask is needed.

In the original prototype, the input sequence length is 64, the prediction length is 3, and the spatial dimension is 64. Therefore, one training sample is constructed from 64 past spatial snapshots and the next 3 future spatial snapshots. If we denote the spatial state at time step $k$by u^k, then one sample can be written as

$$X = [u^1, u^2, \dots, u^{64}], Y = [u^{65}, u^{66}, u^{67}].$$

Here, each u^k is one full spatial snapshot. Since the spatial dimension is 64, each snapshot is a 64-dimensional vector:

$$u^k \in \mathbb{R}^{64}.$$

As a result, the input of one sample is not a single vector, but a matrix:

$$X \in \mathbb{R}^{64\times 64}.$$

This matrix can be interpreted as

$$X = \begin{bmatrix} u^1 \\ u^2 \\ u^3 \\ \vdots \\ u^{64} \end{bmatrix},$$

where each row corresponds to one time step, and each row contains 64 spatial values. In other words, the input matrix stores a short spatiotemporal block: 64 consecutive time steps in the temporal direction, and 64 spatial grid points in the spatial direction.

The target output is smaller, because the model is only asked to predict the next 3 time steps. Therefore, the target can be written as

$$Y \in \mathbb{R}^{3\times 64},$$

or more explicitly,

$$Y = \begin{bmatrix} u^{65} \\ u^{66} \\ u^{67} \end{bmatrix}.$$

So, for one sample, the model takes a 64×64 input matrix and produces a 3×64 output matrix. The meaning is straightforward: use the past 64 spatial states to predict the next 3 spatial states.

This input-output relationship is directly reflected in the dataset code:

```
x = self.data[idx:idx+self.seq_len]
y = self.data[idx+self.seq_len:idx+self.seq_len+self.pred_len]
```

The first line means that x contains seq_len consecutive time steps, which in our case is 64. The second line means that y contains the following pred_len time steps, which in our case is 3. Therefore, these two lines exactly implement the mathematical form

$$X = [u^1, u^2, \dots, u^{64}], \qquad Y = [u^{65}, u^{66}, u^{67}].$$

When batching is used, multiple such samples are stacked together. If the batch size is B, then the input tensor becomes

$$X_{\text{batch}} \in \mathbb{R}^{B \times 64 \times 64},$$

and the target tensor becomes

$$Y_{\text{batch}} \in \mathbb{R}^{B \times 3 \times 64}.$$

For example, if $B = 32$, then one training batch has the shapes

$$X_{\text{batch}} \in \mathbb{R}^{32 \times 64 \times 64}, \; Y_{\text{batch}} \in \mathbb{R}^{32 \times 3 \times 64}.$$

This means that one update step uses 32 samples at the same time; each sample contains 64 past time steps, and each time step is represented by a 64-dimensional spatial vector.

Therefore, the input to the model should not be imagined as one long full trajectory. Instead, it should be understood as a batch of many small matrices, where each matrix represents a short local time window of the KS system. This is the starting point for understanding how the transformer-based model processes the data.

3.2 Self-Attention Mechanism

3.2.1 Input Projection

At the beginning of the self-attention pipeline, the model does not directly use the raw input matrix for attention calculation. Instead, it first maps the raw snapshots into an internal hidden space through a learnable linear projection. This step is necessary because the original input is still written in the physical observation space: each row corresponds to one time step, and each element in that row corresponds to one spatial grid value. Although this representation is physically meaningful, it is not yet the representation used by the attention mechanism.

For one training sample, the input is

$$X \in \mathbb{R}^{64\times 64},$$

where the 64 rows represent 64 consecutive time steps, and each row is a 64-dimensional spatial state. If the i-th row is denoted by x_i, then

$$x_i \in \mathbb{R}^{64}, i = 1,2,\dots,64.$$

Here, x_i is the raw spatial snapshot at one time step. The model then applies the same linear transformation to every row:

$$h_i = x_i W_{\text{in}} + b_{\text{in}}.$$

In this expression,

$x_i \in \mathbb{R}^{64}$ is the raw input vector at time step i,

$W_{\text{in}} \in \mathbb{R}^{64\times d_{\text{model}}}$ is the learnable input projection matrix,

$b_{\text{in}} \in \mathbb{R}^{d_{\text{model}}}$ is the learnable bias vector,

$h_i \in \mathbb{R}^{d_{\text{model}}}$ is the hidden representation after projection.

If the model dimension is chosen as $d_{\text{model}} = 256$, then the dimensions become

$$W_{\text{in}} \in \mathbb{R}^{64\times 256}, b_{\text{in}} \in \mathbb{R}^{256}, h_i \in \mathbb{R}^{256}.$$

So a single time-step vector is transformed from

$$x_i \in \mathbb{R}^{64} \longrightarrow h_i \in \mathbb{R}^{256}.$$

For the whole input matrix, the same operation can be written compactly as

$$H = XW_{\text{in}} + b_{\text{in}},$$

where

$$X \in \mathbb{R}^{64\times 64}, H \in \mathbb{R}^{64\times 256}.$$

Therefore, the input sample keeps its 64 time steps, but each time step is now represented by a 256-dimensional hidden vector instead of the original 64-dimensional physical snapshot.

It is important to note that W_{in} and b_{in} are not given by the dataset. They are model parameters. At the beginning of training, they are initialized automatically, usually with random values. During training, the loss is backpropagated through the network, and these parameters are gradually updated. In this way, the model learns how to transform the raw spatial state into a hidden representation that is more suitable for the later attention operations.

A simple way to understand this step is to compare the raw vector and the hidden vector. The raw vector

$$x_i = [u(x_1, t_i), u(x_2, t_i), \dots, u(x_{64}, t_i)]$$

has a direct physical meaning: each element corresponds to one spatial grid point. After projection, the hidden vector

$$h_i = [h_{i1}, h_{i2}, \dots, h_{i256}]$$

no longer corresponds directly to specific spatial points. Instead, it becomes an internal feature representation used by the model. For example, one hidden component may encode a learned combination of several spatial locations, rather than a single raw measurement. In this sense, the projection step does not create new physical variables; it rewrites the original spatial snapshot into a learned hidden coordinate system.

For example, suppose one raw snapshot is

$$x_i = [0.3, -0.1, 0.5, \ldots]$$

where each number is the value at one spatial grid point. After the projection, it may become

$$h_i = [1.2, -0.7, 0.4, \ldots]$$

with length 256. The new numbers no longer mean "the value at the 7th grid point" or "the value at the 20th grid point." Instead, they are learned internal coordinates produced by the projection matrix. This is why the input projection step is best understood as a re-encoding of the raw snapshot into the model's hidden feature space.

So, the role of projection is clear: it takes the raw input sample

$$X \in \mathbb{R}^{64\times 64}$$

and transforms it into the hidden representation

$$H \in \mathbb{R}^{64\times 256},$$

which then serves as the starting point for the later self-attention calculations. In other words, the logical order is

$$X \to H \to Q, K, V.$$

3.2.2 Positional Encoding

After the input projection step, the raw input sample is transformed into a hidden representation. In our setting, one sample is a 64×64 matrix, and after projection it becomes $H \in \mathbb{R}^{64\times d_{\text{model}}}$. If $d_{\text{model}} = 256$, then $H \in \mathbb{R}^{64\times 256}$.

At this point, the 64 rows still represent 64 consecutive time steps, but each row is now a 256-dimensional hidden vector. However, the transformer still does not naturally know which row comes earlier and which row comes later. This is a key

difference between transformers and recurrent models. In an RNN or LSTM, the sequence is processed step by step, so temporal order is built into the computation itself. In a transformer, the whole sequence is processed together, so order information must be added explicitly. This is exactly why positional encoding is needed.

The basic idea is simple: for each time step, we create a position vector of the same dimension as the hidden representation and then add it to the corresponding hidden vector. Mathematically, this step is written as

$$\widetilde{H} = H + P,$$

where

$$H \in \mathbb{R}^{64\times256}, P \in \mathbb{R}^{64\times256}, \widetilde{H} \in \mathbb{R}^{64\times256}.$$

So for each time step i,

$$\tilde{h}_i = h_i + p_i,$$

where h_i is the projected hidden vector and p_i is the positional encoding vector for that time step. In this way, each row contains both content information and temporal order information.

For example, suppose two time steps have exactly the same hidden representation after Step 1:

$$h_1 = [1.2, -0.7, 0.4, 0.9], \quad h_2 = [1.2, -0.7, 0.4, 0.9].$$

Without positional encoding, these two vectors are identical, so the model cannot distinguish whether they come from different positions in the sequence. Now suppose their positional encoding vectors are

$$p_1 = [0.84, 0.54, 0.01, 0.99995], \quad p_2 = [0.91, -0.42, 0.02, 0.9998].$$

After adding positional encoding, the two vectors become

$$\tilde{h}_1 = h_1 + p_1 \approx [2.04, -0.16, 0.41, 1.89995],$$
$$\tilde{h}_2 = h_2 + p_2 \approx [2.11, -1.12, 0.42, 1.8998].$$

Although the original content vectors are the same, the final vectors are now different because they contain different position information. This is why simple addition is enough to let the model distinguish tokens at different positions.

Positional encoding is not introduced merely to separate vectors that might otherwise be identical. In practice, the hidden vectors at different time steps are usually already different. However, difference in content is not the same as knowledge of order. Even if all hidden vectors are distinct, the transformer still does not naturally know which one comes earlier, which one comes later, or how far apart two positions are in the sequence. The main role of positional encoding is therefore to inject temporal order into the sequence representation. Although it is theoretically possible for two different vectors to become equal after addition, such a case is negligible in practice, especially in high-dimensional hidden spaces. Therefore, positional encoding should be understood as a structured way to provide order information, rather than merely as a tool for distinguishing identical vectors.

Positional encoding does not make temporal order directly readable in a human sense; instead, it injects structured position information into the hidden representations so that the subsequent linear transformations and attention operations can learn to use it.

In the standard transformer, the positional encoding is not learned from data. Instead, it is generated directly by sine and cosine functions:

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right),$$
$$PE(pos, 2i+1) = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right),$$

where pos is the position index in the sequence and i is the dimension index. More precisely, pos tells us which time step we are encoding, while i tells us which pair of dimensions we are using. For example, if $d_{\text{model}} = 4$, then the dimensions are indexed as $0,1,2,3$. In this case:

$i = 0$ corresponds to dimensions 0 and 1,

$i = 1$ corresponds to dimensions 2 and 3.

So the first pair of dimensions uses

$$PE(pos, 0) = \sin(pos), \qquad PE(pos, 1) = \cos(pos),$$

and the second pair uses

$$PE(pos, 2) = \sin\left(\frac{pos}{10000^{2/4}}\right) = \sin\left(\frac{pos}{100}\right),$$
$$PE(pos, 3) = \cos\left(\frac{pos}{10000^{2/4}}\right) = \cos\left(\frac{pos}{100}\right).$$

This makes the role of i much clearer: increasing i means moving to higher dimensions, and higher dimensions vary more slowly across positions.

A small numerical example makes this easier to understand. Suppose the model dimensions were only $d_{\text{model}} = 4$, and we only looked at the first four positions, namely $pos = 0,1,2,3$ ($dt = 0.1,\ t_0 = 0, t_1 = 0.1, t_2 = 0.2, t_3 = 0.3$). Then the positional encoding matrix would have the form

$$P \in \mathbb{R}^{4\times 4}.$$

For position $pos = 0$, we have

$$PE(0,0) = \sin(0) = 0, \qquad PE(0,1) = \cos(0) = 1,$$
$$PE(0,2) = \sin(0/100) = 0, \qquad PE(0,3) = \cos(0/100) = 1.$$

So the first row is

$$PE(0,:) = [0,1,0,1].$$

For position $pos = 1$, we have

$$PE(1,0) = \sin(1) \approx 0.84, \qquad PE(1,1) = \cos(1) \approx 0.54,$$
$$PE(1,2) = \sin(1/100) \approx 0.01 \qquad PE(1,3) = \cos(1/100) \approx 0.99995.$$

So the second row is

$$PE(1,:) \approx [0.84, 0.54, 0.01, 0.99995].$$

For position $pos = 2$, we have

$$PE(2,0) = \sin(2) \approx 0.91, \qquad PE(2,1) = \cos(2) \approx -0.42,$$
$$PE(2,2) = \sin(2/100) \approx 0.02, \quad PE(2,3) = \cos(2/100) \approx 0.9998.$$

So the third row is

$$PE(2,:) \approx [0.91,\ \text{-}0.42, 0.02,\ 0.9998].$$

For position $pos = 3$, we have

$$PE(3,0) = \sin(3) \approx 0.14, \qquad PE(3,1) = \cos(3) \approx -0.99,$$
$$PE(3,2) = \sin(3/100) \approx 0.03, \qquad PE(3,3) = \cos(3/100) \approx 0.99955.$$

So the fourth row is

$$PE(3,:) \approx [0.14, -0.99, 0.03, 0.99955].$$

Here, the notation ":" means "all dimensions in that row." For example, $PE(1,:)$ means the full positional encoding vector at position 1, namely the entire second row of the positional encoding matrix.

Putting all of these rows together, the full positional encoding matrix in this toy example is approximately

$$P \approx \begin{bmatrix} 0 & 1 & 0 & 1 \\ 0.84 & 0.54 & 0.01 & 0.99995 \\ 0.91 & -0.42 & 0.02 & 0.9998 \\ 0.14 & -0.99 & 0.03 & 0.99955 \end{bmatrix}.$$

This matrix makes the idea much clearer: each row corresponds to one position in the sequence, and each row is different. At the same time, different columns behave differently. The first two columns change quickly because they correspond to $i = 0$, while the last two columns change much more slowly because they correspond to $i = 1$. This is exactly how the positional encoding captures both absolute position and relative distance.

In our actual model, the same idea is used with 64 positions and 256 dimensions. Therefore, each of the 64 time steps in the input window is assigned to its own 256-dimensional position vector before self-attention begins. For example, after Step 1, one projected hidden vector may be

$$h_i = [1.2,\ -0.7, 0.4, \dots\ ],$$

and the corresponding positional vector may be

$$p_i = [0.84, 0.54, 0.01, \dots\ ].$$

Then the final representation passed to self-attention is

$$\tilde{h}_i = h_i + p_i.$$

This does not create new physical variables. Instead, it gives each hidden vector a temporal label, so that the transformer can distinguish earlier and later states in the sequence.

The core code is:

```
pe[:, 0::2] = torch.sin(position * div_term)
pe[:, 1::2] = torch.cos(position * div_term)
return x + self.pe[:, :x.size(1), :]
```

The first two lines generate the sine values for the even dimensions and the cosine values for the odd dimensions. The last line adds the positional encoding matrix to the hidden representation. Therefore, the role of this step is clear: before self-attention compares different time steps, the model must first know the order of the sequence. Positional encoding provides this order information in a mathematically structured way. The flow is now

$$X \rightarrow H \rightarrow \widetilde{H} \rightarrow Q, K, V,$$

which means that self-attention operates on the hidden representation after temporal order has been added.

3.2.3 Query, Key, and Value

After the previous two stages, the input sequence has already been transformed into a position-enhanced hidden representation,

$$\widetilde{H} \in \mathbb{R}^{64\times256},$$

where the 64 rows correspond to 64 time steps and the 256 columns correspond to the hidden feature dimension. At this point, each row is no longer the raw spatial state itself. Instead, each row is an internal representation of the full spatial snapshot at one time step. The next question is how self-attention uses this matrix to decide which past time steps are relevant for prediction.

Self-attention does not directly use the same representation for every purpose. Instead, it transforms the same input into three different matrices,

$$Q = \tilde{H}W_Q, \qquad K = \tilde{H}W_K, \qquad V = \tilde{H}W_V,$$

where W_Q, W_K, and W_V are three different learnable parameter matrices. These matrices are not given by the dataset. They are initialized at the beginning of training and then updated through backpropagation. If we first consider a single-head setting and keep the hidden dimension at 256 for simplicity, then

$$W_Q, W_K, W_V \in \mathbb{R}^{256\times256},$$

and therefore

$$Q, K, V \in \mathbb{R}^{64\times256}.$$

So the shapes remain the same, but the meanings are now different.

The reason for introducing three different projections is that self-attention must be performed by two different tasks. First, it must decide which historical states are relevant to the current state. Second, once those relevant states are found, it must decide what useful information should be taken from them. These two tasks are not identical, so the model does not force the same vector to play every role at once. Instead, it learns three different views of the same state.

In general terms, Query describes what kind of historical information the current time step is looking for. Key describes how each historical state can be matched. Value describes what information that historical state can actually provide once it is selected. Instead, the current state uses its Query to compare itself with all historical Keys, and after the relevant historical states are identified, their Values are used to provide the content for the next representation.

To make this idea more concrete in the KS setting, it is helpful to use a simplified physical interpretation. In the real model, the 256 hidden dimensions are learned internal coordinates, so they should not be interpreted too literally as fixed physical quantities. However, for explanation, suppose that a hidden vector roughly summarizes four aspects of the current spatial state:

- overall amplitude,
- strength of spatial gradients,

- oscillation intensity,
- spatial bias or asymmetry.

Then we may write one hidden vector as

$$\tilde{h}_i = [a_i, g_i, o_i, b_i].$$

This is only a teaching analogy, but it helps explain the roles of Q, K, and V.

Suppose that the current state at time step t_{40} has a hidden representation

$$\tilde{h}_{40} = [2.0, 1.5, 0.3, 0.1].$$

This may be interpreted as a state with relatively large amplitude, strong gradients, moderate oscillation, and weak asymmetry. If the model now wants to predict the future, the current state may be especially interested in finding past states with similarly large amplitude and similarly strong gradients. After the query projection, the current state may produce

$$q_{40} = [1.8, 1.2, 0.2, 0].$$

This means that the current state is mainly looking for historical patterns that match strongly in the first two aspects, while the last two aspects are less important for this particular search. In other words, the Query can be understood as a learned description of what kind of past state would be most useful for the current prediction.

Now consider a historical state at time step t_{27}, with hidden representation

$$\tilde{h}_{27} = [1.9, 1.4, 0.4, 0.2].$$

After the key projection, it may become

$$k_{27} = [1.7, 1.3, 0.3, 0.1].$$

This Key does not yet represent the final information to be used by the model. Instead, it acts more like a learned label describing what kind of state t_{27} is. Since both q_{40} and k_{27} emphasize the first two dimensions, the model may regard this historical state as a good match. So, in the KS context, the Key can be understood as the historical state's matchable signature.

For contrast, consider another historical state at time step t_5,

$$\tilde{h}_5 = [0.3, 0.1, 2.0, 1.5]$$

which may correspond to weak amplitude, weak gradients, but strong oscillation and strong asymmetry. Its key may become

$$k_5 = [0.2, 0, 1.9, 1.2].$$

Although this state is also distinctive, it is not the type of state the current Query is mainly looking for. Therefore, it is likely to receive a lower attention score later.

Finally, if a historical state is selected as relevant, the model does not directly copy its Key. Instead, it uses the corresponding Value. For example, the Value of time step t_{27} may be

$$v_{27} = [2.2, 1.0, 0.6, -0.1].$$

This vector represents the actual information that the historical state can contribute once it has been selected. In this sense, the Value is not a matching label, but the information carrier. It is the part that will later be weighed and combined with other selected historical states. Therefore, in this example, the current state uses its Query to search for a past state with a similar dynamical pattern, the historical state uses its Key to show how it can be matched, and its Value provides the actual content that may help the model predict the future. V (Value) means the part of information that the model will finally take from a historical time step and pass forward once that time step is judged to be worth attending to.

It is important to note that the Value matrix is not constructed from the Query and Key matrices. Instead, it is obtained independently from the same input representation through a separate learnable projection. This is because self-attention separates the role of matching from the role of content passing: Queries and Keys determine relevance, while Values carry the actual information to be aggregated.

A toy matrix example makes this even clearer. Suppose the position-enhanced hidden representation is

$$\widetilde{H} = \begin{bmatrix} 1.2 & -0.7 & 0.4 & 0.9 \\ 0.5 & 0.1 & -0.3 & 1.0 \\ -0.2 & 0.8 & 0.6 & -0.4 \\ 1.1 & -0.3 & 0.8 & 0.5 \end{bmatrix}.$$

If three different projection matrices are applied, the resulting query, key, and value matrices may be

$$Q = \begin{bmatrix} 1.2 & -0.7 & 0.4 & 0.9 \\ 0.5 & 0.1 & -0.3 & 1.0 \\ -0.2 & 0.8 & 0.6 & -0.4 \\ 1.1 & -0.3 & 0.8 & 0.5 \end{bmatrix},$$

$$K = \begin{bmatrix} 2.4 & -0.7 & 0.2 & 0.9 \\ 1.0 & 0.1 & -0.15 & 1.0 \\ -0.4 & 0.8 & 0.3 & -0.4 \\ 2.2 & -0.3 & 0.4 & 0.5 \end{bmatrix},$$

$$V = \begin{bmatrix} 2.1 & 0.5 & -0.3 & 1.3 \\ 1.5 & 0.6 & -0.2 & 0.7 \\ -0.6 & 0.6 & 1.4 & 0.2 \\ 1.6 & 0.8 & 0.5 & 1.3 \end{bmatrix}.$$

This toy example shows the essential point: the same input matrix is transformed into three different matrices because self-attention needs three different roles. Query expresses what the current state wants to find, Key expresses how each historical state can be matched, and Value expresses what useful information that historical state can actually contribute.

Therefore, in the KS prediction task, Query, Key, and Value should not be viewed as arbitrary symbols. They provide a structured way for the model to search within the past 64 time steps for dynamically relevant historical patterns, compare the current state with those past patterns, and then prepare the useful historical information for the next stage of attention computation.

3.2.4 Attention Scores, Normalization, and SoftMax

After the query and key matrices have been constructed, the next task is to determine how strongly each time step should attend to every other time step in the input sequence. This is done by comparing queries and keys. For a current time step $i$and a historical time step j, the raw attention score is defined by the dot product

$$s_{ij} = q_i \cdot k_j.$$

This score measures how well the historical state $j$matches what the current state $i$is looking for. If the query and key are well aligned in the hidden space, the score will be large; if they are poorly aligned, the score will be small or even negative.

For the whole sequence, all pairwise scores are computed together in matrix form:

$$S = QK^{\top}.$$

Using the toy matrices from the previous subsection,

$$Q = \begin{bmatrix} 1.2 & -0.7 & 0.4 & 0.9 \\ 0.5 & 0.1 & -0.3 & 1.0 \\ -0.2 & 0.8 & 0.6 & -0.4 \\ 1.1 & -0.3 & 0.8 & 0.5 \end{bmatrix}, \quad K = \begin{bmatrix} 2.4 & -0.7 & 0.2 & 0.9 \\ 1.0 & 0.1 & -0.15 & 1.0 \\ -0.4 & 0.8 & 0.3 & -0.4 \\ 2.2 & -0.3 & 0.4 & 0.5 \end{bmatrix},$$

The raw score matrix is

$$S = \begin{bmatrix} 4.27 & 2.65 & -0.53 & 2.57 \\ 2.97 & 1.59 & -0.85 & 1.51 \\ -1.50 & -0.59 & 0.90 & -0.68 \\ 3.22 & 1.54 & -0.77 & 2.06 \end{bmatrix}.$$

This matrix tells us how strongly each time step matches every other time step. For example, the first row shows how the first time step evaluates all four positions in the toy sequence. A larger number means a stronger raw match.

In standard self-attention, the score matrix is then scaled by the square root of the key dimension:

$$S_{\text{scaled}} = \frac{QK^{\top}}{\sqrt{d_k}}.$$

In this toy example, the key dimension is $d_k = 4$, so $\sqrt{d_k} = 2$. Therefore,

$$S_{\text{scaled}} = \frac{S}{2} = \begin{bmatrix} 2.135 & 1.325 & -0.265 & 1.285 \\ 1.485 & 0.795 & -0.425 & 0.755 \\ -0.75 & -0.295 & 0.45 & -0.34 \\ 1.61 & 0.77 & -0.385 & 1.03 \end{bmatrix}.$$

This scaling step helps keep the numerical values in a stable range. Without it, the dot products may become too large when the hidden dimension is high, which would make the following SoftMax function too sharp.

The scaled score matrix still does not yet represent attention weights. It only tells us which matches are larger or smaller. To convert these scores into actual attention proportions, we apply the SoftMax function row by row:

$$A = \text{softmax}\left(\frac{QK^{\top}}{\sqrt{d_k}}\right).$$

Here, normalization means converting each row of raw scores into a row of positive weights whose sum is equal to 1. After normalization, the entries in each row can be interpreted as attention proportions.

For example, the first row of the scaled score matrix is

$$[2.135, 1.325, -0.265, 1.285].$$

Applying SoftMax means computing

$$softmax(s_1)j = \frac{e^{s_{1j}}}{\sum_{k=1}^{4} e^{s_{1k}}}$$

Numerically, this becomes approximately

$$[0.477, 0.212, 0.043, 0.268].$$

So the first time step assigns about 47.7% of its attention to the first position, 21.2% to the second, 4.3% to the third, and 26.8% to the fourth. The sum of these four weights is equal to 1, which is exactly what normalization means in this context.

Applying the same operation to every row gives the full attention weight matrix:

$$A \approx \begin{bmatrix} 0.477 & 0.212 & 0.043 & 0.268 \\ 0.449 & 0.225 & 0.066 & 0.260 \\ 0.147 & 0.232 & 0.489 & 0.232 \\ 0.456 & 0.197 & 0.062 & 0.285 \end{bmatrix}.$$

Now the matrix is much more useful than the original score matrix. Each row no longer contains arbitrary raw scores. Instead, each row shows how one time step distributes its attention across all positions in the sequence. This is why SoftMax is essential: it turns similarity scores into a normalized attention distribution.

In the KS prediction task, this means that the model is no longer simply asking which historical states are similar to the current one. It is now assigning explicit attention proportions to those historical states. Some past time steps receive higher weights because they are dynamically more relevant to the current state, while others receive lower weights because they are less useful for predicting the future.

At this stage, the model still has not produced the final attention output. It has only determined how strongly each time step should be attended to the others. The result of this stage is the attention weight matrix

$$A \in \mathbb{R}^{64\times 64},$$

which will be used in the next stage to aggregate the value vectors. Therefore, the logical flow so far is

$$X \to H \to \widetilde{H} \to Q, K, V \to QK^{\top} \to \frac{QK^{\top}}{\sqrt{d_k}} \to A.$$

This stage is the part of self-attention where the model converts pairwise similarity into a normalized attention map over the full input sequence.

3.2.5 Attention Output

After the attention weight matrix has been obtained, the next step is to use it to aggregate information from the value matrix. This is the stage where self-attention actually produces a new representation for each time step. The core formula is

$$Z = AV,$$

where A is the attention weight matrix, V is the value matrix, and Z is the output of the self-attention operation.

To understand this formula clearly, it is important to recall the roles of the two matrices. The matrix A tells the model where to look. Each row of A is a normalized attention distribution, so it tells one time step how much attention should be given to all positions in the sequence. The matrix V, on the other hand, tells the model what information to take from each position. Each row of V is the value vector of one time step, that is, the actual content that can be passed forward if that time step is selected.

Therefore, multiplying A and V means the following: for each time step, the model takes all value vectors in the sequence and computes their weighted combination according to the attention proportions in the corresponding row of A. In row-wise form, this can be written as

$$z_i = \sum_{j=1}^{T} a_{ij}\, v_j, \quad \text{(linear combination)}$$

where z_i is the output vector at time step i, a_{ij} is the attention weight assigned by time step i to time step j, and v_j is the value vector at time step j. This equation shows that the output at one position is no longer based only on its own original state. Instead, it becomes a weighted summary of information taken from all time steps in the input window.

Using the toy example from the previous subsection, suppose the attention weight matrix is

$$A = \begin{bmatrix} 0.477 & 0.212 & 0.043 & 0.268 \\ 0.449 & 0.225 & 0.066 & 0.260 \\ 0.147 & 0.232 & 0.489 & 0.232 \\ 0.456 & 0.197 & 0.062 & 0.285 \end{bmatrix},$$

and the value matrix is

$$V = \begin{bmatrix} 2.1 & 0.5 & -0.3 & 1.3 \\ 1.5 & 0.6 & -0.2 & 0.7 \\ -0.6 & 0.6 & 1.4 & 0.2 \\ 1.6 & 0.8 & 0.5 & 1.3 \end{bmatrix}.$$

Then the attention output is

$$Z = AV \approx \begin{bmatrix} 1.81 & 0.62 & 0.11 & 1.09 \\ 1.76 & 0.61 & 0.09 & 1.03 \\ 0.77 & 0.64 & 0.66 & 0.60 \\ 1.76 & 0.62 & 0.16 & 1.10 \end{bmatrix}.$$

This result shows that each output row is a new representation formed by combining the value vectors from all time steps according to the learned attention pattern. For example, the first row of A,

$$[0.477, 0.212, 0.043, 0.268],$$

means that the first time step takes about 47.7% of its information from the first value vector, 21.2% from the second, 4.3% from the third, and 26.8% from the fourth. Therefore, the first output vector in Z is not simply the original state of the first time step. It is a context-enhanced representation obtained by selectively combining information from the whole sequence.

In the KS prediction task, this has a clear physical interpretation. A current time step does not rely only on its own local state when forming its representation. Instead, it can selectively absorb information from other historical time steps that are dynamically relevant. If certain earlier states contain patterns that are important for the future evolution, then their value vectors receive larger weights and contribute more strongly to the output. In this sense, the output matrix Zrepresents the original sequence after contextual information has been injected through attention.

The dimensions are also consistent. If

$$A \in \mathbb{R}^{64\times 64}, V \in \mathbb{R}^{64\times 256},$$

Then $$Z = AV \in \mathbb{R}^{64\times 256}.$$ (linear transformation)

So the output has the same number of time steps as the input, and each time step is still represented in the same hidden dimension. This is important because it allows self-attention to function as a transformation layer inside the model without breaking the sequence structure.

The key point of this stage is therefore simple: the attention matrix tells the model how strongly each time step should attend to all others, while the value matrix tells the model what information each time step can contribute. Their product gives a new sequence representation in which each position has been updated using the most relevant information from the full input window.

Therefore, the logical flow so far is

$$X \to H \to \tilde{H} \to Q, K, V \to QK^{\top} \to \frac{QK^{\top}}{\sqrt{d_k}} \to A \to Z\,.$$

3.2.6 Multi-Head Self-Attention

The explanation above described self-attention in its simplest single-head form. However, the actual transformer model used in this project does not rely on only one attention head. Instead, it uses multi-head self-attention, which means that several attention operations are performed in parallel and then combined. The purpose of this design is not to guarantee that each head has one clearly identifiable physical meaning, but to give the model the ability to learn different attention patterns at the same time. In a complex spatiotemporal system such as the KS equation, a single attention map may not be flexible enough to capture all relevant temporal dependencies.

Suppose the hidden dimension is

$$d_{\text{model}} = 256$$

and the number of attention heads is

$$n_{\text{head}} = 8.$$

Then each head is assigned

$$d_{\text{head}} = \frac{256}{8} = 32$$

dimensions. This means that the full 256-dimensional hidden representation is not processed by one single attention unit. Instead, it is divided into 8 smaller subspaces, and each head learns attention patterns in its own 32-dimensional representation space.

Starting from the same position-enhanced hidden representation

$$\widetilde{H} \in \mathbb{R}^{64\times 256},$$

each head r has its own learnable projection matrices,

$$W_Q^{(r)}, W_K^{(r)}, W_V^{(r)},$$

with dimensions

$$W_Q^{(r)}, W_K^{(r)}, W_V^{(r)} \in \mathbb{R}^{256\times 32}.$$

Therefore, for each head r, the query, key, and value matrices are

$$Q^{(r)} = \widetilde{H}W_Q^{(r)}, \qquad K^{(r)} = \widetilde{H}W_K^{(r)}, \qquad V^{(r)} = \widetilde{H}W_V^{(r)},$$

and their shapes are

$$Q^{(r)}, K^{(r)}, V^{(r)} \in \mathbb{R}^{64\times 32}.$$

So, every head receives the same sequence of 64 time steps, but each head views that sequence through a different learned projection. In other words, the heads do not see different data. They see different learned representations of the same data.

For each head, the attention process is exactly the same as in the single-head case. The attention scores are computed by

$$S^{(r)} = Q^{(r)}(K^{(r)})^{\top},$$

then scaled and normalized,

$$A^{(r)} = \text{softmax}\left(\frac{Q^{(r)}(K^{(r)})^{\top}}{\sqrt{d_{\text{head}}}}\right),$$

and finally used to aggregate the values,

$$Z^{(r)} = A^{(r)}V^{(r)}.$$

Thus, each head produces its own output matrix

$$Z^{(r)} \in \mathbb{R}^{64\times 32}.$$

At this point, the outputs of the different heads still remain separate. To understand what happens next, it is useful to consider a much smaller toy example. Suppose there

are only 2 heads instead of 8, and suppose the total hidden dimension is 6 instead of 256. Then each head receives

$$d_{\text{head}} = \frac{6}{2} = 3$$

dimensions. Assume that the first head produces

$$Z^{(1)} = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \\ 10 & 11 & 12 \end{bmatrix},$$

and the second head produces

$$Z^{(2)} = \begin{bmatrix} 0.1 & 0.2 & 0.3 \\ 0.4 & 0.5 & 0.6 \\ 0.7 & 0.8 & 0.9 \\ 1.0 & 1.1 & 1.2 \end{bmatrix}.$$

Each of these matrices has shape

$$4 \times 3,$$

meaning 4 time steps and 3 output dimensions per head. The next operation is concatenation. Concatenation does not mean averaging or summing the two matrices. It simply means placing them side by side along the feature dimension:

$$Z_{\text{concat}} = \text{Concat}(Z^{(1)}, Z^{(2)}) = \begin{bmatrix} 1 & 2 & 3 & 0.1 & 0.2 & 0.3 \\ 4 & 5 & 6 & 0.4 & 0.5 & 0.6 \\ 7 & 8 & 9 & 0.7 & 0.8 & 0.9 \\ 10 & 11 & 12 & 1.0 & 1.1 & 1.2 \end{bmatrix}.$$

Now the shape becomes

$$4 \times 6.$$

This toy example shows the meaning of concatenation very clearly: each head produces its own partial report, and concatenation simply places these reports together to restore the full feature dimension.

However, concatenation alone is not yet enough. After concatenation, the outputs of different heads are only placed side by side. They have not yet been fully integrated into one unified representation. This is why standard multi-head attention usually introduces one more linear transformation:

$$Z_{\text{out}} = Z_{\text{concat}}\, W_O.$$

Here, W_O is the output projection matrix. Its role is not to produce the final prediction of future time steps. Instead, its role is to mix the information coming from different heads and convert the concatenated result into one unified hidden representation again.

This point is important because concatenation by itself only collects the outputs of different heads. It does not yet allow the model to learn how information from different heads should be combined. The matrix W_O provides exactly this flexibility. It can learn how to strengthen some head outputs, weaken others, and mix useful features across heads. In this sense, the concatenated result is like several separate expert reports placed in one document, while W_O acts like an editor that rewrites them into one coherent final hidden representation.

In the actual model, after concatenating the outputs of 8 heads, we obtain

$$Z_{\text{concat}} \in \mathbb{R}^{64\times 256},$$

and the output projection matrix has dimension

$$W_O \in \mathbb{R}^{256\times 256}.$$

Therefore,

$$Z_{\text{out}} = Z_{\text{concat}} W_O \in \mathbb{R}^{64\times 256}.$$

This means that the output projection does not change the overall hidden size. Instead, it mixes the information from different heads and restores a unified hidden representation.

In the KS forecasting task, the advantage of multi-head attention is not that we can assign a strict physical meaning to each head. In practice, this is usually not possible. Instead, the advantage is that the model is allowed to search for different kinds of temporal dependencies in parallel. One head may become more sensitive to nearby time steps, another may respond more strongly to longer-range similarities, and others

may capture different aspects of the evolving spatiotemporal pattern. Even if these roles cannot be interpreted exactly head by head, the multi-head structure allows the model to represent richer temporal relationships than a single-head design.

Therefore, the main role of multi-head self-attention is to let the model examine the same sequence through several learned subspaces at the same time, then collect these partial results, and finally merge them into a single hidden representation through the output projection matrix W_O. This is why multi-head attention is more expressive than its single-head counterpart.

Therefore, the logical flow so far is

$$X \to H \to \widetilde{H} \to Q^{(r)}, K^{(r)}, V^{(r)} \to \text{softmax}\left(\frac{Q^{(r)}(K^{(r)})^{\top}}{\sqrt{d_{\text{head}}}}\right) \to A^{(r)} \to Z_{concat} \to Z_{out}.$$

3.2.7 Final Output Projection

After the multi-head attention and output projection W_O, the model produces a hidden representation

$$Z_{\text{out}} \in \mathbb{R}^{64\times 256}.$$

This matrix still corresponds to the 64 input time steps, but each time step is now represented in the 256-dimensional hidden space. Therefore, Z_{out} should be understood as an integrated representation of the past sequence, rather than the final prediction.

After the attention mechanism, the model has already gathered information from multiple past time steps. In other words, the hidden vector at a given time step no longer represents only its original local state. Instead, it becomes a weighted combination of information collected from different historical positions. In this sense, the role of attention can be understood as collecting information: it determines which past time steps are most relevant and aggregates their information into the current representation.

However, collecting information does not mean that the representation is already in the best form for prediction. The output of attention is still mainly a weighted sum, that is, a linear combination. A linear combination is good at expressing which features are more important, but it is less effective at expressing which features

become meaningful only when they appear together. For example, in the KS system, a certain future behavior may not be determined by one feature alone, but by the joint presence of two or more features. Suppose that the model needs to detect both a strengthened local wave crest and the appearance of a high-frequency oscillation at the same time. If only a linear combination is used, the model can respond to these two features separately, but it is less able to explicitly represent the stronger relationship that arises when they occur together.

This is exactly why the feedforward network (FFN) is needed. The role of the FFN can be understood as organizing information: attention has already collected useful information from the whole time window, and the FFN further processes that collected information. It does not compare different time steps again. Instead, it applies a nonlinear transformation to the hidden vector at each individual time step. The purpose is to take information that is still only linearly mixed and reorganize, filter, and strengthen it, so that it becomes more suitable for the final prediction.

A simple example makes this clearer. Suppose that after attention, the hidden vector at one time step is

$$z_t = [2.0,\ -1.0, 0.5].$$

This may be roughly interpreted as three aggregated features, such as global amplitude information, local gradient information, and oscillation information. If the model stopped at this stage, then it would only have a linearly mixed representation. Now the first layer of the FFN maps it into a higher-dimensional space, for example

$$h_t = [3.0,\ -2.0, 1.0,\ -0.5].$$

After applying ReLU, we obtain

$$\mathrm{ReLU}(h_t) = [3.0, 0, 1.0, 0].$$

The meaning of this step is not simply that negative numbers are replaced by zero. Rather, it allows the model to selectively preserve some features and suppress others. The second linear layer then projects these filtered and reorganized features back to the original hidden dimension, producing a new representation z_t'. Therefore, the FFN is not just making the vector larger and then smaller again. It uses the higher-dimensional space to further process the relationships between features.

Thus, the division of roles can be stated very directly: attention answers the question of where to collect information from, while the FFN answers the question of how to organize that information after it has been collected. Without attention, the model

cannot effectively gather relevant information from multiple past time steps. Without the FFN, even if the information has been collected, it still remains mainly at the level of a linear combination, which makes it harder for the model to build a more expressive internal representation. This is why a transformer needs not only attention, but also the FFN.

In this project, the model dimension is already fixed as

$$d_{\text{model}} = 256,$$

and the feedforward dimension is also fixed in the code as

$$d_{\text{ff}} = 1024.$$

Therefore, for each time step, the FFN performs the transformation

$$256 \rightarrow 1024 \rightarrow 256.$$

More explicitly, if the hidden representation at one time step is denoted by

$$z_t \in \mathbb{R}^{256},$$

then the first linear layer maps it to a higher-dimensional vector:

$$h_t = z_t W_1 + b_1,$$

where

$$W_1 \in \mathbb{R}^{256\times 1024}, b_1 \in \mathbb{R}^{1024}.$$

As a result,

$$h_t \in \mathbb{R}^{1024}.$$

After this, a nonlinear activation function is applied. A common choice is the Rectified Linear Unit (ReLU), defined as

$$\text{ReLU}(x) = \max(0, x).$$

This means that if a component is positive, it stays unchanged, while if it is negative, it becomes zero. For example,

$$\mathrm{ReLU}(2.3) = 2.3, \mathrm{ReLU}(-1.1) = 0.$$

So if one intermediate hidden vector is

$$h_t = [1.5, -0.8, 2.1, -0.4],$$

then after ReLU it becomes

$$\mathrm{ReLU}(h_t) = [1.5, 0, 2.1, 0].$$

This small example makes the role of ReLU clearer. The nonlinear activation is not added merely for formality. Without it, the entire FFN would still be equivalent to a single linear transformation, even if two linear layers were stacked together. In that case, the model would not gain additional expressive power. ReLU introduces nonlinearity, allowing the model to reshape and refine the features in a way that a purely linear mapping cannot achieve.

The second linear layer then projects the feature dimension back from 1024 to 256:

$$z_t' = \mathrm{ReLU}(h_t)W_2 + b_2,$$

where

$$W_2 \in \mathbb{R}^{1024\times 256}, b_2 \in \mathbb{R}^{256}.$$

Therefore,

$$z_t' \in \mathbb{R}^{256}.$$

So, for one time step, the FFN transformation can be summarized as

$$z_t \in \mathbb{R}^{256} \to h_t \in \mathbb{R}^{1024} \to \mathrm{ReLU}(h_t) \in \mathbb{R}^{1024} \to z_t' \in \mathbb{R}^{256}.$$

If we extend this to the whole sequence, then the matrix dimension changes as

$$64 \times 256 \to 64 \times 1024 \to 64 \times 1024 \to 64 \times 256.$$

It is important to note that the number of time steps remains 64 throughout this process. The FFN does not change the sequence length. It only changes the feature representation at each time step. The same matrices W_1 and W_2, together with the same bias vectors b_1 and b_2, are applied to every row of Z_{out}.

A simple way to understand why the FFN is needed is the following. Suppose that after attention, one time step has already gathered useful information from several past states, so its hidden vector contains many mixed features. The attention mechanism tells the model where to look, but after the information is collected, the model still needs to reorganize and refine these features before making a prediction. The FFN provides exactly this extra processing stage. In other words, attention collects information across time, while the FFN processes and reshapes that information inside each time step.

After the FFN, the model obtains a refined hidden representation, which we may denote by

$$Z_{\text{ffn}} \in \mathbb{R}^{64\times 256}.$$

This representation is still not yet the final prediction. One more projection layer is required to convert the hidden features into the desired output format. Since the goal of the model is to predict the next 3 time steps, and each future time step is a 64-dimensional spatial vector, the final output should have the form

$$Y \in \mathbb{R}^{3\times 64}.$$

Therefore, the final projection layer maps the refined hidden representation to the prediction matrix Y. From an implementation perspective, this can be understood as taking the feature representation after FFN and projecting it into the target prediction space.

So, the role of each stage can now be summarized clearly. The attention mechanism extracts and organizes temporal information from the past 64 time steps. The FFN further refines the hidden features at each time step through a nonlinear transformation. The final projection layer then converts the refined hidden representation into the prediction of the next 3 time steps.

Therefore, the logical flow up to this point becomes

$X \to H \to \widetilde{H} \to Q^{(r)}, K^{(r)}, V^{(r)} \to \text{softmax}\left(\frac{Q^{(r)}(K^{(r)})^T}{\sqrt{d_{\text{head}}}}\right) \to A^{(r)} \to Z_{\text{concat}} \to Z_{\text{out}} \to Z_{\text{ffn}} \to Y.$

From an implementation perspective, this operation is straightforward. The model takes the encoded representation and applies an output projection, for example:

output = self.output_projection(x_encoded)

and then reshapes the result into

$$(32, 3, 64)$$

The key idea is that the attention mechanism is responsible for extracting and organizing information from the past 64 time steps, while the final projection layer converts this information into the prediction of the next 3 time steps.

3.3 Easy Attention Mechanism

3.3.1 Core Idea of Easy Attention

Easy Attention should not be understood as a completely different model from standard self-attention. Its overall purpose remains the same: the model still needs to assign weights across different time positions and then aggregate information accordingly. The key difference is much more specific. In standard self-attention, the attention matrix is generated dynamically from query-key matching; in Easy Attention, the attention matrix itself is learned directly as a parameter. Sanchis-Agudo et al. explicitly argue that, for temporal prediction of chaotic systems, queries, keys, and even the SoftMax operation are not strictly necessary for obtaining useful attention scores, and they therefore propose to treat the attention scores themselves as learnable parameters.

Using the notation of this report, the standard self-attention pipeline for one head can be written as

$$\widetilde{H} \to Q, K, V \to A = \text{softmax}\left(\frac{QK^{\top}}{\sqrt{d_{\text{head}}}}\right) \to Z = AV.$$

If

$$\widetilde{H} \in \mathbb{R}^{64\times 256}, d_{\text{model}} = 256, n_{\text{head}} = 8,$$

then each head has dimension

$$d_{\text{head}} = \frac{256}{8} = 32.$$

For one head r, the value projection still has the form

$$V^{(r)} = \widetilde{H}W_V^{(r)}, W_V^{(r)} \in \mathbb{R}^{256\times 32}, V^{(r)} \in \mathbb{R}^{64\times 32}.$$

In this work, the multi-head structure is preserved in Easy Attention. For example, with a total dimension of 256 and 8 heads, each head still operates on 32 dimensions, consistent with Section 3.2.6.

In standard self-attention, this value matrix is combined with a dynamically computed attention matrix

$$A^{(r)} \in \mathbb{R}^{64\times 64},$$

which comes from the query-key similarity. In Easy Attention, this is the exact point where the mechanism changes. The query and key branches are removed, and the attention matrix is replaced by a directly learned matrix

$$\alpha^{(r)} \in \mathbb{R}^{64\times 64}.$$

Therefore, the computation for each head becomes

$$V^{(r)} = \widetilde{H}W_V^{(r)}, \qquad Z^{(r)} = \alpha^{(r)}V^{(r)}, \qquad Z^{(r)} \in \mathbb{R}^{64\times 32}.$$

This shows that most of the dimensional flow remains unchanged. The multi-head structure is still preserved, the value path is still preserved, and the output is still obtained by weighting and aggregating value vectors. What changes is only the source of the weighting matrix. In this sense, Easy Attention is best viewed as a re-parameterization of one internal step of the attention mechanism rather than a replacement of the whole architecture. This interpretation is fully consistent with the

original paper, where the authors state that they remove W_Q, W_K, and softmax, and directly learn αinstead.

A small toy example makes this structure clearer. Suppose the sequence length is 4, the total hidden dimension is 6, and the number of heads is 2. Then each head has dimension 3. For one head, the standard self-attention computation would produce

$$Q, K, V \in \mathbb{R}^{4\times 3}, A = \text{softmax}(QK^{\top}) \in \mathbb{R}^{4\times 4}, Z = AV \in \mathbb{R}^{4\times 3}.$$

In Easy Attention, the same head still keeps

$$V \in \mathbb{R}^{4\times 3},$$

but the dynamically generated attention matrix is replaced by

$$\alpha \in \mathbb{R}^{4\times 4},$$

so that the output becomes

$$Z = \alpha V \in \mathbb{R}^{4\times 3}.$$

Thus, the head still mixes information across time using a 4×4 weighting matrix and still produces a 4×3 output. The only essential modification is that the weighting matrix is no longer computed from $QK^{\top}$, but learned directly.

The original paper provides an especially useful way to interpret this weighted aggregation. In their motivating example, the attention scores behave like a probability distribution over candidate trajectories, and the final output can be written as an expected value. Using the notation of this report, the same idea can be expressed as

$$\hat{y}_i = E(Z_i) = \sum_{j=1}^{m} \alpha_{i,j}\, v_j,$$

where v_j is the value vector contributed by the j-th historical time step, and $\alpha_{i,j}$is the attention weight assigned by the current output position $i$to that historical position. This means that the model does not simply choose one past state. Instead, it forms the output by taking a weighted combination of several past value vectors. Sanchis-Agudo et al. explicitly interpret this as a weighted integration over the past

time, and they emphasize that the αmatrix highlights temporal dependencies, while $W_V$contains the dependence on the spatial inputs.

To make this interpretation concrete, suppose three candidate value vectors are

$$v_1 = [1,2], \qquad v_2 = [3,0], \qquad v_3 = [2,4],$$

and the learned attention weights for the current output are

$$\alpha_{i,1} = 0.2, \qquad \alpha_{i,2} = 0.5, \qquad \alpha_{i,3} = 0.3.$$

Then the output is

$$\hat{y}_i = 0.2\, v_1 + 0.5\, v_2 + 0.3\, v_3 = 0.2[1,2] + 0.5[3,0] + 0.3[2,4] = [2.3,1.6].$$

This example shows that Easy Attention still performs the same essential action as standard attention: it aggregates information across multiple historical positions through a weighted sum. The novelty is not in abandoning attention, but in learning the temporal weighting pattern α directly.

The deeper motivation given in the paper is also important for this project. The authors argue that, in chaotic dynamical systems, the numerical input values may vary strongly from one sample to another, while the underlying system dynamics remain the same. Under this viewpoint, recomputing an input-dependent attention map for every sample may be less meaningful than directly learning an important pattern that encodes the system dynamics themselves. Their Easy Attention formulation is therefore optimized during training but becomes input-independent at inference time, so that the learned attention matrix reflects temporal dependency structure rather than instantaneous attention of the raw input values. This idea is especially relevant for the KS system, where the observed states evolve chaotically, but the governing dynamics remain fixed.

For this reason, the core idea of Easy Attention can be summarized very simply. It does not change the global objective of attention, and it does not remove the value-based information aggregation mechanism. Instead, it keeps the overall pipeline largely intact and modifies only the attention-weight generation step. Standard self-attention learns how to compute the attention matrix from query-key similarity, whereas Easy Attention learns the attention matrix itself. That is the central idea of the method, and it is precisely this change that makes the mechanism both structurally simpler and conceptually appealing for chaotic temporal prediction.

3.3.2 Why Easy Attention Is Simpler

Sanchis-Agudo et al. explicitly state that their method removes W_Q, W_K, and softmax, and that this reduces the computational complexity from $4n_i^2$ to $2n_i^2$. The intuitive meaning is straightforward: in standard self-attention, the model takes a detour through extra projections and similarity computation in order to obtain A; in Easy Attention, the temporal weighting pattern is learned directly as a parameter, so the computational path becomes shorter.

The difference between standard self-attention and Easy Attention can be clearly understood by looking at how many "full pairwise interaction steps" are performed, and what matrices are actually multiplied.

We use the same setting as in this work: sequence length $n = 64$. The attention matrix is therefore 64×64, which means there are 4096 pairwise interactions in total.

In standard self-attention, this matrix is not directly learned. Instead, it is constructed through a sequence of operations. The core computations can be traced as follows.

First, the input is multiplied by a projection matrix to produce a query representation. This is one full matrix multiplication involving all time steps. Then, the input is multiplied again to produce a key representation. This is the second full matrix multiplication.

After that, the model performs pairwise comparison: the query matrix is multiplied by the transpose of the key matrix. This is the step where every time step interacts with every other time step. For $n = 64$, this produces $64 \times 64 = 4096$ comparisons. This is the third major operation.

Finally, the resulting matrix is normalized (SoftMax), which is applied over all 4096 elements. Although this is not a matrix multiplication, it still processes the entire $n \times n$structure and therefore counts as another full operation over the same scale.

So if we summarize only the dominant steps that operate over all pairwise interactions, we have:

- input multiplied to produce Q → one large operation
- input multiplied to produce K → one large operation
- Q multiplied with $K^\top$→ full pairwise interaction (4096 terms when $n = 64$)
- normalization over the 64×64 matrix → another full pass

This is why the cost is summarized as proportional to:

$$4n^2$$

The key point is that the model goes through an entire "build → compare → normalize" pipeline before it even uses the attention weights.

In Easy Attention, this pipeline is shortened.

The attention matrix is no longer constructed through comparisons. Instead, it is directly learned as a parameter. This removes the need to generate Q and K and completely removes the pairwise comparison step.

Now the computation only involves two main steps.

First, the input is multiplied once to produce the value representation. This is one large matrix multiplication over all time steps.

Second, the learned attention matrix (of size 64×64) is directly multiplied with the value matrix. This is the only step where all time steps interact, producing again 4096 weighted combinations.

So in Easy Attention, the dominant operations are:

- input multiplied to produce V → one large operation
- learned matrix multiplied with V → full interaction (4096 terms when $n = 64$)

There is no Q, no K, and no pairwise comparison between them.

Therefore, the total number of major operations is reduced to:

$$2n^2$$

To make this more concrete, we compare the two cases using $n = 64$:

Standard self-attention:

$$4 \times 64^2 = 16384$$

Easy Attention:

$$2 \times 64^2 = 8192$$

The reduction does not come from changing the size of the attention matrix. The matrix is still 64×64, and still represents 4096 relationships. The reduction comes from eliminating the process of recomputing these relationships through Q–K comparisons.

In other words, standard self-attention spends computation to "figure out" the relationships at every forward pass, while Easy Attention directly learns these relationships and applies them. This is why the number of full interaction steps drops from four to two.

This is especially meaningful for chaotic temporal prediction. In systems such as KS, the numerical values may vary strongly from one input window to another, but the underlying dynamical law remains the same. Sanchis-Agudo et al. therefore argue that, for such systems, it may be more meaningful to learn an important pattern that encodes the system dynamics rather than recomputing the instantaneous attention of the input values for every sample. Their words are that the model learns "an importance score for the input variables that encodes the system dynamics rather than the instantaneous attention of the input values." Hence, Easy Attention is simpler not only because it reduces the number of operations, but also because it replaces a sample-dependent weighting process with a direct parameterization of temporal dependency structure.

The experiments reported in the paper support the claim that this simplification does not merely trade accuracy for speed. The authors report that Easy Attention reduces self-attention complexity by about 45%, saves about 15% computation time during training, and achieves lower error on several chaotic-system benchmarks. They further conclude that the easy-attention-based transformer has lower computational cost, less complexity, and fewer parameters than its self-attention counterpart.

Chapter 4. Training Strategy and Optimization

4.1 Learning Objective and Loss Function

After the model structure has been defined, the next question is no longer how the matrices are arranged, but what the model is actually trying to learn. In the original

prototype, the model receives the first 64 time steps and predicts the next 3 time steps. The true values of these 3 future steps are already known in the dataset, but they are not given to the model as input. They are only used after the prediction is made, so that the model output can be compared with the correct answer. In the notation already used in this report, one sample has input shape 64×64 and target shape 3×64, and if the batch size is 32, then both the prediction and the target have shape $32 \times 3 \times 64$.

The model therefore needs a numerical rule that tells it whether its prediction is good or bad. This rule is called the loss function. In this project, the most natural choice is the mean squared error (MSE), because both the prediction and the target are matrices of real numbers. If the model prediction is denoted by $\hat{Y}$ and the true target is denoted by Y, then the loss can be written as

$$\mathcal{L} = \frac{1}{N}\sum(\hat{Y} - Y)^2,$$

where N is the total number of scalars (number of elements) values being averaged. If the batch size is 32, the prediction length is 3, and the spatial dimension is 64, then

$$N = 32 \times 3 \times 64.$$

This formula means the following. First, compute the difference between the prediction and the true value at every position. Second, square each difference so that all errors become positive. Third, average all of them into one number. If this final number is small, the prediction is close to the truth. If it is large, the prediction is poor.

To make this process easier to understand, it is helpful to pause the real KS model for a moment and use a much smaller teaching example. Suppose we build a tiny network with only two input numbers, two hidden variables, and one final output. This network is not the real model used in the project; it is only a toy example to show clearly how a prediction and a loss are computed.

Let the input be

$$x = [1, 2],$$

and let the true target be

$$y = 1.$$

Because the input has 2 numbers, the input dimension is 2. Suppose we also choose a hidden dimension of 2. Then the first weight matrix must have shape 2×2, because it maps a 2-dimensional input to a 2-dimensional hidden representation. Let

$$W_1 = \begin{bmatrix} 0.1 & 0.2 \\ 0.3 & 0.4 \end{bmatrix}, b_1 = [0.1, 0.1].$$

Now suppose the hidden layer is mapped to a single scalar output. Since the hidden layer has 2 numbers and the output has 1 number, the second weight matrix must have shape 2×1. Let

$$W_2 = \begin{bmatrix} 0.5 \\ 0.6 \end{bmatrix}, \qquad b_2 = 0.2.$$

The tiny network computes

$$h = xW_1 + b_1, \hat{y} = hW_2 + b_2.$$

Substituting the numbers gives

$$h = [1, 2] \begin{bmatrix} 0.1 & 0.2 \\ 0.3 & 0.4 \end{bmatrix} + [0.1, 0.1] = [0.8, 1.1],$$

and then

$$\hat{y} = [0.8, 1.1] \begin{bmatrix} 0.5 \\ 0.6 \end{bmatrix} + 0.2 = 1.26.$$

Now we compare it with the true value $y = 1$. To keep the derivation simple, let the loss function be the scalar squared error

$$L = \frac{1}{2}(\hat{y} - y)^2. \ (\frac{1}{2} \text{ is just for easy calculation})$$

Substituting the numbers gives

$$L = \frac{1}{2}(1.26 - 1)^2 = \frac{1}{2}(0.26)^2 = 0.0338.$$

So for this single training example, the final loss is

$$L = 0.0338.$$

This small example already contains the full idea of training. The network starts from some matrices W_1 and W_2, produces a prediction $\hat{y}$, compares it with the true answer y, and then reduces the whole error to one number L. This number is the quantity the optimizer tries to reduce. In other words, all parameter updates in later steps are guided by one simple principle: change the matrices in such a way that the loss becomes smaller.

This is exactly the same idea in the real KS model. The only difference is scale. In the toy example, the prediction is one scalar, while in the actual project the prediction is a 3×64 matrix for each sample. But the training objective is identical in both cases: compute the prediction, compare it with the true target, and use the loss function to measure the error.

In the actual MSE computation, the prediction is compared with the ground truth element by element. This means that each predicted scalar is subtracted from the true scalar at the same sample index, the same future time-step index, and the same spatial index. If the prediction and the target are denoted by

$$\hat{Y}, Y \in \mathbb{R}^{32\times3\times64}$$

then the loss is computed from the differences

$$\hat{Y}_{b,t,s} - Y_{b,t,s}$$

where b denotes the sample index in the batch, t denotes the predicted future time step, and s denotes the spatial position. After these differences are computed, each of them is squared, and then all of them are averaged. Therefore, the MSE can be written more explicitly as

$$\mathcal{L} = \frac{1}{32 \times 3 \times 64} \sum_{b=1}^{32} \sum_{t=1}^{3} \sum_{s=1}^{64} (\hat{Y}_{b,t,s} - Y_{b,t,s})^2$$

This expression shows clearly that the loss is not computed by comparing whole vectors at once in an abstract way. Instead, it is obtained by comparing every predicted number with its corresponding true number at the same location in the output tensor and then averaging all these squared errors into one final scalar loss value.

The next subsection will explain how this error determines the direction in which the matrices should move.

4.2 Gradient Descent: Why Parameters Move

Once the loss has been computed, the next question is how the model should change its parameters in order to reduce that loss. The key point is that the trainable quantities in the network are not the input x and not the target y, but the parameters themselves, such as the weight matrices and bias vectors. Therefore, if the prediction is wrong, the only way to improve the model is to change these parameters.

For this reason, the model must answer a very specific question for every parameter: if this parameter changes slightly, how will the loss change? This is exactly what the derivative tells us. For example, if we compute

$$\frac{\partial L}{\partial W_2},$$

this does not mean "apply a formal rule to a matrix" in an abstract way. It means: how sensitive is the loss to each element inside W_2? Every entry of this gradient matrix corresponds to one entry of W_2, and tells us whether increasing that entry would increase the loss or decrease it.

To make this precise, we continue the toy example from Section 3.4.1:

$$h = [0.8, 1.1], \qquad \hat{y} = 1.26, \qquad y = 1, \qquad L = \frac{1}{2}(\hat{y} - y)^2.$$

The first derivative is

$$\frac{\partial L}{\partial \hat{y}} = \hat{y} - y = 1.26 - 1 = 0.26.$$

This number already has a direct practical meaning. Since it is positive, increasing $\hat{y}$ would increase the loss, while decreasing $\hat{y}$ would reduce the loss. In other words, the current prediction is too large.

Now consider the second-layer weight matrix

$$W_2 = \begin{bmatrix} 0.5 \\ 0.6 \end{bmatrix}.$$

The prediction depends on W_2 through

$$\hat{y} = hW_2 + b_2.$$

So the loss does not depend on W_2 directly, but through $\hat{y}$. This is why the chain rule is needed:

$$\frac{\partial L}{\partial W_2} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial W_2}.$$

From the forward equation, $\hat{y}$ is linear in W_2, so

$$\frac{\partial \hat{y}}{\partial W_2} = \begin{bmatrix} 0.8 \\ 1.1 \end{bmatrix}.$$

Therefore,

$$\frac{\partial L}{\partial W_2} = 0.26 \begin{bmatrix} 0.8 \\ 1.1 \end{bmatrix} = \begin{bmatrix} 0.208 \\ 0.286 \end{bmatrix}.$$

This result should be read carefully. It means:

- If the first entry of W_2 increases slightly, the loss will increase at a local rate of about 0.208;
- If the second entry of W_2 increases slightly, the loss will increase at a local rate of about 0.286.

So both entries of W_2 are currently pushing the loss upward. That is why both of them should be reduced.

The same idea applies to the bias term b_2. Since

$$\hat{y} = hW_2 + b_2,$$

we have

$$\frac{\partial L}{\partial b_2} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial b_2} = 0.26 \times 1 = 0.26.$$

Again, this means that increasing b_2 would increase the loss, so $b_2$should also be reduced.

At this point, we can already see the practical meaning of the sign of a gradient:

If $\frac{\partial L}{\partial \theta} > 0$, then increasing θ would increase the loss, so θ should move downward.

If $\frac{\partial L}{\partial \theta} < 0$, then increasing θ would decrease the loss, so θ should move upward.

This is the real reason why the update rule uses subtraction:

$$\theta \leftarrow \theta - \eta \frac{\partial L}{\partial \theta}.$$

The derivative points toward increasing loss, so subtracting it moves the parameter in the direction of decreasing loss.

The learning rate η plays a different role. It does not determine the direction of motion; the gradient already does that. Instead, η determines the step size. In other words, it answers the question: once the correct direction is known, how far should the parameter move in that direction? A large learning rate makes the step larger, while a small learning rate makes the step more cautious.

For example, if

$$\eta = 0.1,$$

then the updated second-layer parameters are

$$W_2^{\text{new}} = \begin{bmatrix} 0.5 \\ 0.6 \end{bmatrix} - 0.1 \begin{bmatrix} 0.208 \\ 0.286 \end{bmatrix} = \begin{bmatrix} 0.4792 \\ 0.5714 \end{bmatrix},$$

and

$$b_2^{\text{new}} = 0.2 - 0.1(0.26) = 0.174.$$

These numbers show clearly what “parameter movement” means. The parameters do not move randomly, and they do not move because of a fixed hand-designed correction. They move because the gradients tell the model how strongly each parameter contributes to the current loss, and gradient descent then pushes the parameters in the opposite direction.

Exactly the same logic applies to every other trainable matrix in the full model. Whether the parameter is an input projection matrix, a value projection matrix, an Easy Attention weight matrix, or a final output projection, the meaning is always the same: compute how the loss changes with respect to that parameter, then update the parameter in the direction that reduces the loss.

4.3 Backpropagation Through the Model

In Section 3.4.2, the meaning of gradient descent was explained: once the gradient of the loss with respect to a parameter is known, that parameter can be updated in the direction of lower loss. However, for earlier layers in the network, the gradient is not obtained directly. Instead, it must be passed backward through the intermediate variables. This is the essential idea of backpropagation.

We continue the same toy example:

$$x = [1, 2], \qquad h = [0.8, 1.1], \qquad \hat{y} = 1.26, \qquad y = 1.$$

The forward relations are

$$h = xW_1 + b_1, \qquad \hat{y} = hW_2 + b_2, \qquad L = \frac{1}{2}(\hat{y} - y)^2.$$

From Section 3.4.2, we already know

$$\frac{\partial L}{\partial \hat{y}} = 0.26.$$

This gradient is the starting point of the backward pass. But the first-layer parameters W_1 and b_1 do not appear in $\hat{y}$ directly. They first influence the hidden vector h, then $h$influences $\hat{y}$, and finally $\hat{y}$ influences L. Therefore, before computing the gradients of $W_1$and b_1, the error must first be transferred from $\hat{y}$ back to h.

This is why the next quantity to compute is

$$\frac{\partial L}{\partial h}.$$

Using the chain rule,

$$\frac{\partial L}{\partial h} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial h}.$$

Now look carefully at

$$\hat{y} = hW_2 + b_2.$$

Since

$$h = [h_1, h_2], W_2 = \begin{bmatrix} 0.5 \\ 0.6 \end{bmatrix},$$

the scalar output can be written as

$$\hat{y} = 0.5h_1 + 0.6h_2 + b_2.$$

So the derivatives of $\hat{y}$ with respect to the hidden variables are

$$\frac{\partial \hat{y}}{\partial h_1} = 0.5, \qquad \frac{\partial \hat{y}}{\partial h_2} = 0.6.$$

Therefore,

$$\frac{\partial \hat{y}}{\partial h} = [0.5, 0.6] = W_2^{\top}.$$

This is the first important appearance of a transpose. In the forward pass, the hidden row vector h is multiplied by the column vector W_2. In the backward pass, the error must be distributed from the single output back to the two components of the hidden layer. That is why the same weights appear again, but now in transposed form. The transpose reflects the reversal of the information flow.

Substituting the numbers gives

$$\frac{\partial L}{\partial h} = 0.26[0.5, 0.6] = [0.13,0.156].$$

So the error has now reached the hidden layer.

The next step is to transfer this hidden-layer error back to the first-layer parameters. Recall that

$$h = xW_1 + b_1.$$

Since W_1 affects the loss through h, the chain rule gives

$$\frac{\partial L}{\partial W_1} = x^\top \frac{\partial L}{\partial h}.$$

This formula is also worth understanding carefully. The input is

$$x = [1,2], x^\top = \begin{bmatrix} 1 \\ 2 \end{bmatrix},$$

and the hidden-layer gradient is

$$\frac{\partial L}{\partial h} = [0.13, 0.156].$$

Therefore,

$$\frac{\partial L}{\partial W_1} = \begin{bmatrix} 1 \\ 2 \end{bmatrix} [0.13, 0.156] = \begin{bmatrix} 0.13 & 0.156 \\ 0.26 & 0.312 \end{bmatrix}.$$

The transpose of x appears here for the same basic reason: the gradient of a matrix must have exactly the same shape as the matrix itself. Since $W_1 \in \mathbb{R}^{2\times 2}$, its gradient must also be a 2×2 matrix. The input x is a row vector of shape 1×2, while $\frac{\partial L}{\partial h}$ is also a row vector of shape 1×2. If x were used directly, the multiplication would not produce the correct shape. By transposing x, we obtain a column vector of shape 2×1, and then $(2 \times 1)(1 \times 2) = (2 \times 2)$, which matches the shape of W_1.

So the transpose is not added by convention. It appears because the backward computation must produce a gradient with the same dimensions as the original parameter matrix.

The same logic applies to the first-layer bias. Since b_1 is added directly to h, we have

$$\frac{\partial L}{\partial b_1} = \frac{\partial L}{\partial h} = [0.13, 0.156].$$

Up to this point, in the toy example, we have already computed the gradients $\frac{\partial L}{\partial \hat{y}}$, $\frac{\partial L}{\partial W_2}$, $\frac{\partial L}{\partial b_2}$, $\frac{\partial L}{\partial h}$, $\frac{\partial L}{\partial W_1}$, and $\frac{\partial L}{\partial b_1}$. At this point, the entire backward path of the toy network is visible:

$$L \rightarrow \hat{y} \rightarrow h \rightarrow W_1, b_1.$$

This is the central idea of backpropagation. The loss is first differentiated with respect to the final output, then the error is passed backward to the hidden variables, and only after that can the gradients of earlier parameters be computed. Each step uses the chain rule, and each transpose appears because the backward pass must respect the dimensions of the original forward computation.

In the real KS model, the same idea remains true. The network contains many more matrices and many more intermediate variables, but the logic is unchanged: the error is computed at the end, then passed backward layer by layer, and finally converted into gradients for all trainable parameters.

4.4 How the Matrices Are Learned

The next question is therefore very natural: where do these matrices actually come from, and why do they eventually become meaningful?

The answer is that the matrices are not manually constructed from physical formulas, and they are not assigned fixed numerical values in advance. Instead, they are treated as trainable parameters. At the beginning of training, the model only knows the structure of the computation, such as which matrix multiplies the input and which matrix connects the hidden layer to the output. However, the numerical values inside these matrices are not yet meaningful. They are usually initialized automatically, often as small random numbers.

This is exactly what happened in the toy example. The matrices

$$W_1 = \begin{bmatrix} 0.1 & 0.2 \\ 0.3 & 0.4 \end{bmatrix}, \quad W_2 = \begin{bmatrix} 0.5 \\ 0.6 \end{bmatrix}$$

and the biases

$$b_1 = [0.1, 0.1], \qquad b_2 = 0.2$$

were simply taken as the current parameter values of the model. In a real training procedure, such numbers usually come from initialization rather than from human design. Their role at the beginning is only to make the forward computation possible. They are not expected to be correct at the start.

What makes them meaningful is the repeated training process. For one sample, the model first uses the current matrices to produce a prediction. In our toy example, that prediction was

$$\hat{y} = 1.26.$$

This prediction was then compared with the true target

$$y = 1,$$

which gave the loss

$$L = 0.0338.$$

After that, backpropagation provided the following gradients:

$$\frac{\partial L}{\partial \hat{y}} = 0.26, \qquad \frac{\partial L}{\partial W_2} = \begin{bmatrix} 0.208 \\ 0.286 \end{bmatrix}, \qquad \frac{\partial L}{\partial b_2} = 0.26,$$

$$\frac{\partial L}{\partial h} = [0.13, 0.156], \qquad \frac{\partial L}{\partial W_1} = \begin{bmatrix} 0.13 & 0.156 \\ 0.26 & 0.312 \end{bmatrix}, \qquad \frac{\partial L}{\partial b_1} = [0.13, 0.156].$$

These gradients are the key quantities that make learning possible. Each of them tells the model how the loss would change if the corresponding parameter increased slightly. Therefore, they provide directions for correction.

For example, in the toy example the gradient of the second-layer weight matrix was

$$\frac{\partial L}{\partial W_2} = \begin{bmatrix} 0.208 \\ 0.286 \end{bmatrix}.$$

Both entries are positive. This means that if the entries of W_2 were increased, the loss would increase. Therefore, these entries should be reduced. With learning rate

$$\eta = 0.1,$$

the updated matrix becomes

$$W_2^{\text{new}} = W_2 - \eta \frac{\partial L}{\partial W_2} = \begin{bmatrix} 0.5 \\ 0.6 \end{bmatrix} - 0.1 \begin{bmatrix} 0.208 \\ 0.286 \end{bmatrix} = \begin{bmatrix} 0.4792 \\ 0.5714 \end{bmatrix}.$$

The same happens for the bias:

$$b_2^{\text{new}} = 0.2 - 0.1(0.26) = 0.174.$$

And similarly, the first-layer parameters are updated as

$$W_1^{\text{new}} = W_1 - \eta \frac{\partial L}{\partial W_1}, \qquad b_1^{\text{new}} = b_1 - \eta \frac{\partial L}{\partial b_1}.$$

This is the essential meaning of learning. The matrices are not "discovered" all at once. They are corrected little by little. At each training step, the model predicts, compares, computes gradients, and updates. Then it repeats the same process again with the new parameter values. Over many iterations, the matrices gradually move away from their random initial values and become increasingly adapted to the prediction task.

This point is especially important for understanding the larger KS model. In the full model, the trainable matrices are no longer only W_1 and W_2. Instead, they include the input projection matrix, the query, key, and value matrices, the output projection matrix after multi-head attention, the directly learned Easy Attention matrix α, and the final output projection layer. Although these matrices are larger and more numerous, they are learned in exactly the same way as in the toy example. None of them is manually assigned a meaningful final value. They all start from initialization, and then the training process gradually shapes them through loss computation, backpropagation, and gradient descent.

The whole process is parameters → prediction →loss → gradient → new parameters

In practice, convergence does not mean that every parameter must keep decreasing or that no oscillation is allowed. Parameters in neural-network training may increase, decrease, or oscillate, especially under mini-batch updates. What matters is whether the updates become progressively more stable, whether the loss approaches a relatively steady range, and whether the model outputs stop changing drastically from one training stage to another. Therefore, "convergence" in this context should be understood as stabilization of the training dynamics rather than monotonic decrease of individual parameter values. Training is usually stopped not when every gradient becomes exactly zero, but when the loss no longer decreases significantly, the validation performance no longer improves, or the predefined number of epochs has been reached.

4.5 Adam Optimizer (Adaptive Moment Estimation)

In this project, parameter updating is performed using the Adam optimizer. In the code, it is defined as

optimizer = optim.Adam(model.parameters(), lr=config.lr)

This means that all trainable parameters in the model are updated by Adam during training.

The initial learning rate is set to $\eta = 1 \times 10^{-4}$. Mini-batch training is used in this project with batch size 32, meaning that 32 samples are fed into the model at once to compute a single loss and its corresponding gradients for one parameter update.

However, the basic gradient descent does not always guarantee convergence. Consider the one-dimensional loss function

$$L(w) = w^2.$$

Its derivative is

$$\frac{dL}{dw} = 2w.$$

If standard gradient descent is used, the update rule is

$$w_{t+1} = w_t - \eta \cdot 2w_t = (1 - 2\eta)w_t,$$

where η is the learning rate. The behavior of training now depends directly on the value of η.

If

$$0 < \eta < 0.5,$$

then

$$| 1 - 2\eta | < 1,$$

so the parameter gradually moves toward zero and the loss decreases. For example, if $w_0 = 1$and $\eta = 0.1$, then

$$w_1 = 0.8, w_2 = 0.64, w_3 = 0.512,$$

so the parameter converges steadily.

If

$$\eta = 0.75,$$

then

$$w_{t+1} = -0.5w_t.$$

Starting from $w_0 = 1$, we obtain

$$w_1 = -0.5,\ w_2 = 0.25, w_3 = -0.125.$$

In this case, the parameter oscillates around zero, but the amplitude becomes smaller, so the method still converges.

However, if

$$\eta = 1.1,$$

then

$$w_{t+1} = -1.2w_t.$$

Starting again from $w_0 = 1$, we obtain

$$w_1 = -1.2, w_2 = 1.44, w_3 = -1.728.$$

Now the parameter not only oscillates, but the oscillation becomes larger and larger. As a result, the loss also increases instead of decreasing. This is a clear example of divergence.

Therefore, basic gradient descent may diverge when the learning rate is too large. Mild oscillation is not necessarily a problem, because it may still shrink and converge. But if the step size is too aggressive, the parameter can repeatedly overshoot the minimum, and the oscillation may grow instead of decay. This is one reason why more stable optimization methods such as Adam are often preferred in practice.

Under this setting, Adam is preferred over basic gradient descent because it smooths

the noisy gradients from each mini-batch and adaptively scales the update size, leading to more stable and efficient training.

To understand what Adam does, it is helpful to connect it directly with Section 3.4. In Section 3.4, we derived gradients such as

$$\frac{\partial L}{\partial W_2}, \frac{\partial L}{\partial b_2}, \frac{\partial L}{\partial W_1}, \frac{\partial L}{\partial b_1}.$$

These gradients tell us how the loss changes with respect to each parameter. In the simplest gradient descent method, one would update a parameter θ directly by

$$\theta \leftarrow \theta - \eta \frac{\partial L}{\partial \theta}.$$

This rule uses only the current gradient. In other words, it looks only at the slope at the current step and then moves the parameter in the opposite direction.

However, this simple rule has a practical weakness. In real training, the gradient may fluctuate from step to step. Sometimes it becomes larger, sometimes smaller, and sometimes it changes direction slightly. If the optimizer trusts only the current gradient, the update may become unstable. This is exactly the problem that Adam tries to improve.

For this reason, Adam does not use the raw gradient alone. Instead, for each parameter, it keeps track of two extra quantities.

The first quantity is

$$g_t = \frac{\partial L}{\partial \theta},$$

which is simply the current gradient at step t. This is the same gradient derived by backpropagation in Section 3.4. By itself, g_t already contains two kinds of information:

Its sign tells us the direction of change,

Its magnitude tells us how sensitive the loss is to that parameter.

But Adam does not use g_t directly. Instead, it constructs a smoothed version of the gradient:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t.$$

Here, m_t is the running average of recent gradients. It can be understood as a more stable estimate of the update direction. The constant β_1 is a number between 0 and 1, usually close to 1. Its role is to decide how much past information should be remembered. If β_1 is large, then the optimizer relies more on the recent history; if β_1 is smaller, it relies more on the current gradient.

A simple numerical example makes this clearer. Suppose the gradients in three consecutive steps are

$$g_1 = 0.2, \qquad g_2 = 0.3, \qquad g_3 = 0.25,$$

and let

$$\beta_1 = 0.9.$$

Starting from $m_0 = 0$, we obtain

$$\begin{aligned} m_1 &= 0.9 \times 0 + 0.1 \times 0.2 = 0.02, \\ m_2 &= 0.9 \times 0.02 + 0.1 \times 0.3 = 0.048, \\ m_3 &= 0.9 \times 0.048 + 0.1 \times 0.25 = 0.0682. \end{aligned}$$

So instead of using only the current gradient 0.25, Adam uses a smoothed value that also reflects the recent history. It is used because a moving average reduces oscillation and makes the update direction more stable.

The second extra quantity in Adam is

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2.$$

This is the running average of the squared gradients. Here v_t does not represent direction. Instead, it estimates the typical size of recent gradients. The square is used because we want to measure magnitude only. For example, gradients 0.2 and -0.2 have opposite directions, but they have the same size, and after squaring they both become 0.04.

Again, β_2 is a decay factor between 0 and 1, usually very close to 1. It controls how much past gradient magnitude is remembered. If $\beta_2 = 0.999$, the estimate changes very smoothly over time.

In practice, Adam is usually used with the PyTorch default coefficients $\beta_1 = 0.9$ and $\beta_2 = 0.999$, while the first- and second-moment estimates are initialized as $m_0 = 0$ and $v_0 = 0$.

For example, if

$$g_t = 0.2,$$

then

$$g_t^2 = 0.04.$$

If we start from $v_0 = 0$, then

$$v_1 = 0.999 \times 0 + 0.001 \times 0.04 = 0.00004.$$

So v_t tells the optimizer whether the gradient for that parameter has recently been small or large.

Once m_t and v_t are available, Adam updates the parameter by

$$\theta \leftarrow \theta - \eta \frac{m_t}{\sqrt{v_t} + \epsilon}.$$

This is the core Adam formula. Each term has a clear role:

m_t: gives a smoothed update direction,

$\sqrt{v_t}$: reflects the recent size of the gradient,

η: is the overall learning rate,

ϵ: is a very small positive constant used only for numerical stability, so that the denominator never becomes zero.

The reason for dividing by $\sqrt{v_t}$ is also practical rather than mysterious. If the gradients of a parameter have been consistently large, then v_t becomes large, so the denominator becomes large, and the update step is automatically reduced. If the gradients have been small, then the denominator remains smaller, and the parameter can still move effectively. In this sense, Adam automatically adjusts the step size for each parameter.

This can be seen in a simple example. Suppose that for one parameter,

$$m_t = 0.2, v_t = 0.04, \eta = 0.1.$$

Then the update becomes

$$\theta_{\text{new}} = \theta - 0.1 \cdot \frac{0.2}{\sqrt{0.04}} = \theta - 0.1 \cdot \frac{0.2}{0.2} = \theta - 0.1.$$

Now consider another case where the gradient magnitude has been much larger, so that

$$m_t = 0.2, v_t = 1.$$

Then the update becomes

$$\theta_{\text{new}} = \theta - 0.1 \cdot \frac{0.2}{1} = \theta - 0.02.$$

So even though the direction is similar, the actual update becomes much smaller. This shows exactly why Adam is more stable than basic gradient descent: it does not treat all parameters in the same way but adapts the effective step size according to the recent gradient magnitude.

4.6 Learning Rate Decay

In neural-network training, the learning rate does not need to remain constant throughout the entire process. In the early stage of training, the model parameters are usually far from an optimal solution, so a relatively larger learning rate helps the loss decrease faster. In the later stage, however, if the learning rate remains unchanged, parameter updates may oscillate around a good region and make further improvement difficult. Therefore, it is common to gradually reduce the learning rate during training, which is known as learning rate decay.

In this project, the learning rate is not manually adjusted at fixed epochs. Instead, it is controlled automatically by a scheduler. In the code, the following scheduler is used:

```
scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer, patience=10, factor=0.5)
```

and after each validation phase, the scheduler is updated by

scheduler.step(avg_val_loss).

Here, the optimizer should be understood as the component that converts gradients into actual parameter updates. The loss function measures prediction error, backpropagation computes gradients, and the optimizer applies a specific update rule (in this case Adam) to modify the model parameters. Learning rate decay works together with the optimizer by adjusting the step size during training.

It is important to note that learning rate decay is not triggered when the validation loss reaches a fixed absolute value (for example, 0.01 or 0.001). Instead, it is triggered when the validation loss fails to show sufficient improvement over time.

More specifically, the scheduler monitors the validation loss after each epoch and keeps track of the best value observed so far. If the validation loss does not improve beyond this best value for a number of consecutive epochs, the learning rate is reduced. In this project, $parameter$ patience =10 means that the scheduler waits for 10 consecutive epochs without clear improvement before reducing the learning rate. The parameter factor =0.5 means that once triggered, the learning rate is multiplied by 0.5.

A numerical example can clarify this mechanism. Suppose the validation loss in early epochs is $0.120, 0.108, 0.101, 0.099, 0.098,$
which indicates continuous improvement. In this case, the learning rate remains unchanged because each epoch produces a better result than before.

Now suppose the best validation loss has reached $0.0980,$ and the following epochs produce values such as $0.0981, 0.0980, 0.0982, 0.0981, 0.0980. ..$

Although these values fluctuate slightly, they do not show a clear improvement over the current best value*. If this situation continues for 10 consecutive epochs, the scheduler interprets it as a plateau. At that point, the learning rate is reduced:

$$\eta \leftarrow 0.5\eta.$$

For example, if the initial learning rate is

$$\eta_0 = 10^{-4},$$

then after the first decay it becomes

$$\eta_1 = 5 \times 10^{-5},$$

and after another decay it becomes

$$\eta_2 = 2.5 \times 10^{-5}.$$

It is also important to note that the "no improvement" counter is reset whenever a sufficiently better validation loss is observed. For instance, if after several stagnant epochs the validation loss drops to a noticeably lower value (e.g., from 0.0980 to 0.0978), then this is treated as a true improvement, and the scheduler restarts its patience counting from zero.

Therefore, learning rate decay in this project is driven by validation performance rather than by a fixed schedule. Its role is to adapt the step size to different stages of training: a larger learning rate allows faster progress in the early stage, while a smaller learning rate enables more precise adjustments once the model reaches a plateau. This makes the training process both efficient and stable.

*Best value:

By default, PyTorch uses a relative threshold of 10^{-4}. Therefore, if the current best validation loss is denoted by best, then a new validation loss is counted as a true improvement only when new loss $<$ best $\times (1 - 10^{-4})$.
For example, if best $= 0.0980$, then the new validation loss must be smaller than $0.0980 \times (1 - 10^{-4}) = 0.0979902$. In order to reset the patience counter. Otherwise, the scheduler treats the result as no significant improvement. If this happens for 10 consecutive epochs, the learning rate is reduced by a factor of 0.5.

4.7 Gradient Clipping

During neural-network training, the magnitude of parameter updates is directly determined by the gradients. If some gradients become excessively large, a single update step may cause drastic changes in the parameters, leading to unstable training or even divergence. This issue is more likely to occur in deep models or sequence-based tasks.

To address this problem, **gradient clipping** is applied in this project. In the code, it is implemented as

clip_grad_norm_(model.parameters(), max_norm=1.0).

The core idea of gradient clipping is to **limit the overall magnitude of the gradients**. Specifically, it computes the global norm (typically the L2 norm) of all gradients:

$$\| \mathbf{g} \| = \sqrt{g_1^2 + g_2^2 + \cdots + g_n^2}.$$

If this norm exceeds the predefined threshold (here 1.0), all gradients are rescaled proportionally:

$$\mathbf{g} \leftarrow \mathbf{g} \cdot \frac{1.0}{\| \mathbf{g} \|}.$$

If the norm is already below the threshold, no modification is applied.

A simple numerical example illustrates this process. Suppose the gradients of two parameters are

$$g_1 = 3,\ g_2 = 4.$$

Then the L2 norm is

$$\| \mathbf{g} \| = \sqrt{3^2 + 4^2} = 5.$$

Since $5 > 1.0$, clipping is applied. The scaling factor is

$$\frac{1.0}{5} = 0.2.$$

The updated gradients become

$$g_1 = 0.6,\ g_2 = 0.8.$$

In this way, the direction of the gradient is preserved, while its magnitude is reduced.

Therefore, gradient clipping does not change the optimization direction, but it prevents excessively large updates in a single step. In this project, where mini-batch training may introduce variability in gradients across batches, gradient clipping helps ensure stable training and reduces the risk of divergence.

4.8 Validation Monitoring and Best Model Selection

During training, it is not sufficient to evaluate model performance only on the training data. A model may achieve a low training loss while performing poorly on unseen data. Therefore, in this project, validation data is used to monitor model performance throughout the training process.

After each epoch, the model is evaluated on the validation set, and the average validation loss is computed as

$$\text{avg_val_loss}.$$

This value is then compared with the best validation loss observed so far, denoted as best_loss. If the current validation loss is smaller, the model parameters are saved as the new best checkpoint.

This process can be summarized as

$$\text{if avg_val_loss} < \text{best_loss, then save model.}$$

It is important to note that this strategy is not equivalent to early stopping. The training process continues for the full number of epochs. However, instead of using the model from the final epoch, the model corresponding to the lowest validation loss is selected as the final result.

The purpose of this strategy is to ensure that the selected model has the best generalization performance on unseen data. Even if the training loss continues to decrease, the validation loss may start to increase after a certain point, indicating that the model is no longer improving in terms of generalization. By keeping track of the best validation performance, the model used for evaluation is less affected by such fluctuations.

Integrated Example of Parameter Update

To illustrate how the training components in this section work together, consider a simplified example where a model contains a parameter matrix

$$W = \begin{bmatrix} 1.0 & -1.0 \\ 0.5 & 2.0 \end{bmatrix}.$$

Suppose that after a forward pass and loss computation, backpropagation produces the gradient matrix

$$G = \frac{\partial L}{\partial W} = \begin{bmatrix} 3 & -4 \\ 0 & 1 \end{bmatrix}.$$

Step 1: Gradient Clipping

First, gradient clipping is applied. The global L2 norm of the gradient is

$$\| G \| = \sqrt{3^2 + (-4)^2 + 0^2 + 1^2} = \sqrt{26} \approx 5.10.$$

Since this exceeds the threshold 1.0, all gradients are scaled by

$$\frac{1.0}{5.10} \approx 0.196.$$

The clipped gradient becomes

$$G' = \begin{bmatrix} 0.588 & -0.784 \\ 0 & 0.196 \end{bmatrix}.$$

Step 2: Adam Update

Assume that the previous Adam states are

$$M_{t-1} = \begin{bmatrix} 0.2 & -0.1 \\ 0 & 0.3 \end{bmatrix}, V_{t-1} = \begin{bmatrix} 1.0 & 1.0 \\ 1.0 & 1.0 \end{bmatrix}.$$

Using $\beta_1 = 0.9,\ \beta_2 = 0.999$, we compute

$$M_t = 0.9M_{t-1} + 0.1G'$$
$$= \begin{bmatrix} 0.2388 & -0.1684 \\ 0 & 0.2896 \end{bmatrix}$$
$$V_t = 0.999V_{t-1} + 0.001(G')^2$$
$$\approx \begin{bmatrix} 0.9993 & 0.9996 \\ 0.9990 & 0.9990 \end{bmatrix}.$$

Step 3: Parameter Update

With learning rate

$$\eta = 10^{-4},$$

the parameter update is

$$W_{\text{new}} = W - \eta \frac{M_t}{\sqrt{V_t}}.$$

Since $\sqrt{V_t} \approx 1$, the update is approximately

$$W_{\text{new}} \approx W - 10^{-4} \cdot M_t.$$

Step 4: Learning Rate Decay (Across Epochs)

The above update is performed at every training step. However, the learning rate η is not fixed across the entire training process. If the validation loss does not improve for 10 consecutive epochs, the learning rate is reduced:

$$\eta \leftarrow 0.5\eta.$$

This reduces the update magnitude in later stages of training.

Step 5: Model Selection

At the end of each epoch, the validation loss is evaluated. If a new minimum is observed, the corresponding model parameters are saved. After training, the model with the lowest validation loss is selected as the final model.

Chapter 5. Prediction and Evaluation Using NRMSE

5.1 Prediction Procedure: Short-Term and Autoregressive Forecasting

After the model has been trained, the next question is no longer how the parameters are updated, but how the trained model is actually used to produce forecasts. In this

project, prediction is performed in two different modes: short-term prediction and long-term autoregressive prediction.

For one prediction sample, the model receives the past 64 time steps as input. Since each time step is a 64-dimensional spatial snapshot, the input can be written as

$$X = \begin{bmatrix} u(t_1) \\ u(t_2) \\ \vdots \\ u(t_{64}) \end{bmatrix} \in \mathbb{R}^{64\times 64},$$

where each row $u(t_i) \in \mathbb{R}^{64}$ is one spatial state of the KS system at a fixed time. The model then outputs its prediction for the next 3 time steps,

$$\hat{Y} = \begin{bmatrix} \hat{u}(t_{65}) \\ \hat{u}(t_{66}) \\ \hat{u}(t_{67}) \end{bmatrix} \in \mathbb{R}^{3\times 64}.$$

At this stage, the model is no longer being trained. No gradient is computed, no backpropagation is performed, and no parameter is updated. The trained model is used purely as a forecasting tool.

Short-term prediction evaluates the model under ideal conditions. The input window consists entirely of true data, and the model predicts only the next 3 steps, namely $\hat{u}(t_{65}), \hat{u}(t_{66}), \hat{u}(t_{67})$, which are directly compared with the corresponding true states $u(t_{65}), u(t_{66}), u(t_{67})$. This reflects the model's ability to learn the local mapping from past to near-future states.

To evaluate long-term behavior, autoregressive prediction is used. The model is first applied to a true input window $[u(t_1), \dots, u(t_{64})]$to produce $[\hat{u}(t_{65}), \hat{u}(t_{66}), \hat{u}(t_{67})]$. These predicted values are then fed back into the input by removing the earliest three time steps and appending the predictions, forming a new input window $[u(t_4), \dots, u(t_{64}), \hat{u}(t_{65}), \hat{u}(t_{66}), \hat{u}(t_{67})]$. The same procedure is repeated, so that in the next step the model produces $[\hat{u}(t_{68}), \hat{u}(t_{69}), \hat{u}(t_{70})]$, and the window is updated again. Continuing this process, the input window gradually contains more predicted values and fewer true values, and the model must rely on its own outputs to continue forecasting.

A simplified implementation is shown below:

```
window = true_data[:64]
predictions = []
```

```
for step in range(num_steps):
    pred = model(window)    # shape (3, 64)
    predictions.append(pred)
    window = torch.cat([window[3:], pred], dim=0)
```

5.2 NRMSE Calculation and Interpretation

This section documents how the original prototype evaluated prediction accuracy using RMSE and an instantaneous-state NRMSE. The final metric is defined in Section 6.2.

In general form, for a vector-valued prediction $y^{\text{pred}} \in \mathbb{R}^N$ and the corresponding true value $y^{\text{true}} \in \mathbb{R}^N$, the RMSE is defined as

$$\text{RMSE} = \sqrt{\frac{1}{N}\sum_{i=1}^{N}\left(y_i^{\text{pred}} - y_i^{\text{true}}\right)^2}.$$

This measures the absolute magnitude of the prediction error. However, RMSE depends on the scale of the data. For systems like the KS equation, where the amplitude of the spatial field may vary over time, this makes RMSE less suitable for comparing prediction quality across different time steps.

To better understand why RMSE alone is not sufficient, consider a simple numerical example. Suppose that at a certain time step the true state has a relatively small amplitude, for example

$$u^{\text{true}} = [1,1,1,1],$$

and the model predicts

$$u^{\text{pred}} = [0,0,0,0].$$

In this case, the error at each component is 1, so the RMSE is equal to 1. Now consider another situation where the true state has a much larger amplitude,

$$u^{\text{true}} = [100,100,100,100],$$

and the prediction is

$$u^{\text{pred}} = [99,99,99,99].$$

Here, the error at each component is still 1, so the RMSE is again equal to 1. However, the two situations are clearly very different. In the first case, the prediction is completely wrong, while in the second case, the prediction is very accurate. As a result, the same RMSE value can correspond to very different levels of prediction quality.

NRMSE resolves this issue by normalizing the RMSE with respect to the magnitude of the true signal. In the first case, the signal magnitude is approximately 1, so the NRMSE is

$$\text{NRMSE} \approx \frac{1}{1} = 1,$$

which correctly indicates a large error. In the second case, the signal magnitude is approximately 100, so the NRMSE becomes

$$\text{NRMSE} \approx \frac{1}{100} = 0.01,$$

which reflects a very small relative error. Therefore, NRMSE provides a scale-independent measure of prediction accuracy and allows meaningful comparison across different time steps and different regimes of the system.

RMSE is normalized by the magnitude of the true signal, leading to the NRMSE:

$$\text{NRMSE} = \frac{\sqrt{\frac{1}{N}\sum_{i=1}^{N}\left(y_i^{\text{pred}} - y_i^{\text{true}}\right)^2}}{\sqrt{\frac{1}{N}\sum_{i=1}^{N}\left(y_i^{\text{true}}\right)^2}}.$$

In this project, each spatial state has dimension $N = 64$, so the formula becomes

$$\mathrm{NRMSE}(t) = \frac{\sqrt{\frac{1}{64}\sum_{i=1}^{64}\left(u_i^{\mathrm{pred}}(t) - u_i^{\mathrm{true}}(t)\right)^2}}{\sqrt{\frac{1}{64}\sum_{i=1}^{64}\left(u_i^{\mathrm{true}}(t)\right)^2}}.$$

In practice, the model outputs three future time steps at each forward pass, and NRMSE is computed separately for each predicted time step. The NRMSE curve is then constructed by evaluating all predicted time steps along the rollout trajectory. It is important to note that no NRMSE is defined for the first 64 time steps, since these correspond to true input data. The first NRMSE value appears at t_{65}, which is the first predicted time step.

Therefore, NRMSE is a **relative error**, not an absolute error. This is why it is more interpretable than RMSE when comparing prediction quality over time or across different cases.

A simple numerical example makes this clearer. Suppose the true state at one prediction time is

$$u^{\mathrm{true}}(t) = [2,\ 2,\ 2,\ 2],$$

and the predicted state is

$$u^{\mathrm{pred}}(t) = [2,\ 2,\ 2,\ 2].$$

Then the error at every component is zero, so

$$\mathrm{RMSE}(t) = 0, \mathrm{NRMSE}(t) = 0.$$

This is the ideal case. It means the prediction is exactly equal to the true state.

Now consider another case:

$$u^{\mathrm{true}}(t) = [2,\ 2,\ 2,\ 2], u^{\mathrm{pred}}(t) = [0,\ 0,\ 0,\ 0].$$

The component wise error is always -2, so

$$\mathrm{RMSE}(t) = \sqrt{\frac{1}{4}(4 + 4 + 4 + 4)} = 2.$$

The RMS magnitude of the true state is also

$$\sqrt{\frac{1}{4}(4+4+4+4)} = 2.$$

Therefore,

$$\text{NRMSE}(t) = \frac{2}{2} = 1.$$

So NRMSE = 1 means that the error magnitude is about as large as the true signal magnitude itself. This is already a large error. The prediction is not completely meaningless, but it is no longer close in a strict quantitative sense.

Now consider a third case. Let the predicted state differ from the true state in such a way that the error energy is twice the true signal energy. Then the numerator becomes $\sqrt{2}$ times the denominator, so

$$\text{NRMSE}(t) = \sqrt{2} \approx 1.414.$$

This corresponds to:

- Error energy ≈ twice the signal energy
- Prediction is essentially decorrelated from truth

In KS literature, this is often used as a **failure threshold**.

This behavior is closely related to the chaotic nature of the KS system. In chaotic systems, small errors grow approximately exponentially:

$$\delta(t) = \delta_0 e^{\lambda t},$$

where λ is the Lyapunov exponent. The corresponding characteristic time scale is the **Lyapunov time:**

$$T_L = \frac{1}{\lambda}.$$

In this project, time is measured in discrete steps rather than continuous time. If each step corresponds to a time interval Δt, then the error growth can be written as

$$\delta(k) = \delta_0 e^{\lambda k \Delta t},$$

where k is the number of time steps. This shows that the exponential growth still exists but is now expressed in terms of discrete steps. An important point is that reducing the time step Δt does not change the intrinsic chaotic behavior of the system. The Lyapunov exponent λ remains the same. However, when Δt becomes smaller, the factor $\lambda \Delta t$ also becomes smaller. As a result, the error appears to grow more slowly when observed per step. This does not mean that the system becomes less chaotic. Instead, the same exponential growth is distributed over a larger number of steps.

A simple numerical example makes this clearer. Suppose $\lambda = 1$, the total physical time is 1, and the initial error is $\delta_0 = 1$. If we take one large step with $\Delta t = 1$, then after one step the error becomes

$$\delta = e^1 \approx 2.718.$$

Now consider the same physical time interval but divide it into 10 smaller steps with $\Delta t = 0.1$. In that case, the error growth per step is

$$e^{0.1} \approx 1.105.$$

So after the first step the error becomes 1.105, after the second step it becomes 1.105^2, and after 10 steps it becomes

$$(1.105)^{10} \approx 2.718.$$

Therefore, the final error growth over the full physical time interval is the same. The only difference is that when Δt is smaller, the growth is spread across more discrete steps, so the error appears to increase more slowly on a step-by-step plot.

At the beginning of prediction, the error is small because the model starts from true data. As time progresses, the error grows approximately exponentially due to the

chaotic nature of the system. Eventually, the NRMSE approaches a value close to $\sqrt{2}$, at which point the prediction is no longer correlated with the true trajectory. That was the prototype's illustrative horizon. The final experiments use thresholds 0.5 and 1, not $\sqrt{2}$, and report VPT in Lyapunov times as defined in Section 6.2. This horizon is not determined only by the model itself but is fundamentally limited by the Lyapunov time of the underlying chaotic system.

The original illustrative horizon was defined as

$$t_{\text{fail}} = \min\{t : \text{NRMSE}(t) \geq \sqrt{2}\}.$$

This belongs to the original illustrative convention. Section 6.2 gives the criterion used for every result in the revised experiments.

A simple implementation for computing NRMSE is shown below:

```
def compute_nrmse(pred, true):
    mse = ((pred - true)**2).mean(dim=1)
    rmse = torch.sqrt(mse)
    norm = torch.sqrt((true**2).mean(dim=1))
    return rmse / norm
```

Chapter 6. Downsampling Results and Discussion

6.1 What changed after the original prototype

I have kept the preceding derivations and code examples because they record how I first understood the KS problem. They describe a 64-step input, a 3-step target, an instantaneous-state normalization of error, and validation-loss checkpoint selection. Those examples are not the settings used for the results below. In the early inference script, only the first of the three outputs was fed back, so the block-rollout sketch in Chapter 5 is an illustration rather than a record of that run. The final runs were built after I found that several choices could make a short-horizon result look stronger than its autonomous forecast.

The most consequential data error in the earlier workflow was to generate heavily overlapping windows and split those windows at random. Two nominally different samples could then share nearly all their time states across training and validation. A small validation loss in that setting did not establish performance on a new trajectory. In the revised KS runs, I split 100 simulated trajectories by trajectory identity with seed 42: 80 for training and 20 held out for validation. Windows are made separately inside each partition; they never cross a trajectory boundary. The 20 trajectories reported here are validation trajectories, not an independent test set.

The current KS simulation uses a periodic interval of length 22 and a saved time interval of 0.1 system-time units. Its configured duration is 10,000 time units, with a numerical warm-up before retained states. Spatial sampling takes every ds-th point of the original 64-point state, so ds = 16 leaves four observed channels and ds = 32 leaves two. This is spatial sparsification. It does not multiply the time interval by 16 or 32, and the forecasts concern only those observed channels; no 64-point reconstruction is claimed.

For Self Attention and Easy Attention + SGU, the final input window contains 128 states and the supervised target contains the next 100. At inference, all 100 predicted states are appended and the window advances by 100 before the next model call. This aligns the output used at training with the output used in rollout. The LSTM instead predicts one state at a time and feeds each prediction back. It uses 128 known states to initialize prediction at ds = 16 and 256 at ds = 32, with closed-loop curriculum horizons of 10, 20, 50 and 100 during training. Consequently this is a comparison of completed pipelines, not a controlled change of architecture alone.

The final attention runs use AdamW, batch size 32, an initial learning rate of 0.0001 and a cosine schedule. The final LSTM runs use Adam, batch size 64 and an initial learning rate of 0.0005. The reported checkpoints are the archived best-rollout checkpoints, selected using held-out rollout behavior rather than the smallest one-step validation loss alone. This is another reason that the earlier optimization discussion should be read as the prototype history, not as a description of every final run.

6.2 Error definition and valid prediction time

For each observed spatial channel j, I compute a mean and population standard deviation from the retained training trajectories only, after spatial sampling and before overlapping windows are generated. Thus the divisor in the variance is the number of training snapshots M (ddof = 0). The same training statistics standardize model input and target. For evaluation, predictions and truth are first put back in their original physical scale; their difference is then divided once by the fixed training standard deviation for each channel. All evaluated channels have a standard deviation above 10^{-8}, so the implemented numerical lower bound of 10^{-8} does not change the reported values.

$$e_i(k) = \left[\frac{1}{D} \sum_{j=1}^{D} \left(\frac{\hat{u}_{i,j}(k) - u_{i,j}(k)}{\sigma_j^{\text{train}}} \right)^2 \right]^{1/2}$$

Here i identifies one held-out trajectory, k is a predicted step after the last known true input, D is four or two for the two KS settings, and the training standard deviation of channel j is the fixed standard deviation of channel j. This is the root mean square of channel-standardized errors. It is not the original prototype formula that divides by the instantaneous RMS of the true state, and it is not a single global standard deviation applied after taking the spatial RMSE. I compute the curve for each trajectory first, then plot the mean of the 20 curves with the 10th–90th percentile band. The band describes variation among trajectories; it is not a confidence interval or repeated training-seed uncertainty.

I define a first-crossing step separately for each unsmoothed trajectory curve. Starting with the first genuinely predicted state as k = 1, I record the first k whose error reaches or exceeds the specified threshold. I use threshold 1 as the main criterion and 0.5 as a stricter companion. I then multiply each first-crossing step by the saved physical interval, 0.1, and by the reference Lyapunov exponent 0.043 used in the KS evaluation configuration. Thus one forecast step corresponds to 0.0043 Lyapunov times. This exponent is a configured reference, not a quantity measured anew from these runs. All 20 trajectories cross both thresholds within 3000 predicted steps, so no right-censoring adjustment is needed. The reported mean VPT averages the 20 converted first-crossing times; it need not equal the first crossing of the mean NRMSE curve. A finite VPT records only the initial period before an error threshold is first crossed; it is not evidence of accurate prediction over the rest of the rollout. The NRMSE figures keep forecast steps on their horizontal axes, while VPT values below are in Lyapunov times.

$$k^*_{a,i} = \min\{k \geq 1 : e_i(k) \geq a\}, \quad \text{VPT}^{(\lambda)}_{a,i} = \lambda_{\text{ref}} \Delta t \, k^*_{a,i}, \quad a \in \{0.5, 1\}$$

The NRMSE plots mark both 1 and $\sqrt{2}$. The second line is only a statistical reference: an expected squared error near 2 can arise if prediction and truth are nearly uncorrelated, similarly centered, and have comparable variance under the same fixed normalization. Those conditions are not assured for a finite set of 20 trajectories. Bias, changing forecast amplitude, residual correlation and sampling variation can move the late mean above or below $\sqrt{2}$. It is not the VPT threshold, and hovering near it does not mean a forecast remains accurate. When a curve exceeds $\sqrt{2}$, I report that directly rather than forcing a saturation story.

6.3 Results at ds = 16: four observed channels

The three individual curves below use the same held-out set, 3000-step forecast horizon and 0.1 system-time interval. Each shows the mean trajectory error, a 10th–90th percentile band, and horizontal references at 1 and $\sqrt{2}$. They expose both the typical growth and the spread that a single average can hide.

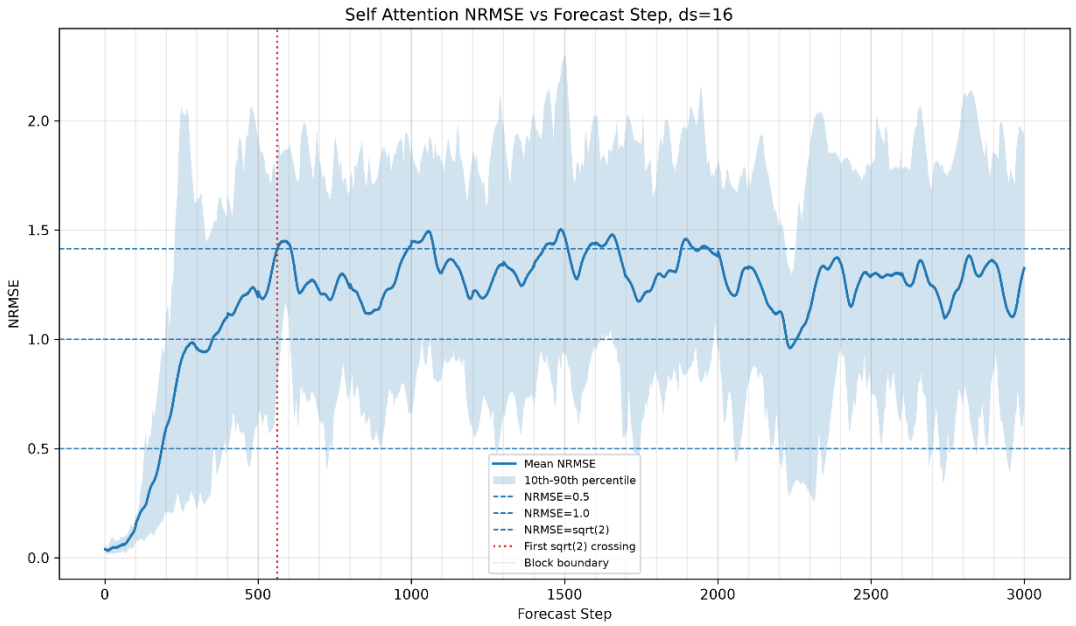


Figure 6.1. Self Attention, ds = 16. Mean NRMSE and 10th–90th percentile across 20 held-out validation trajectories.

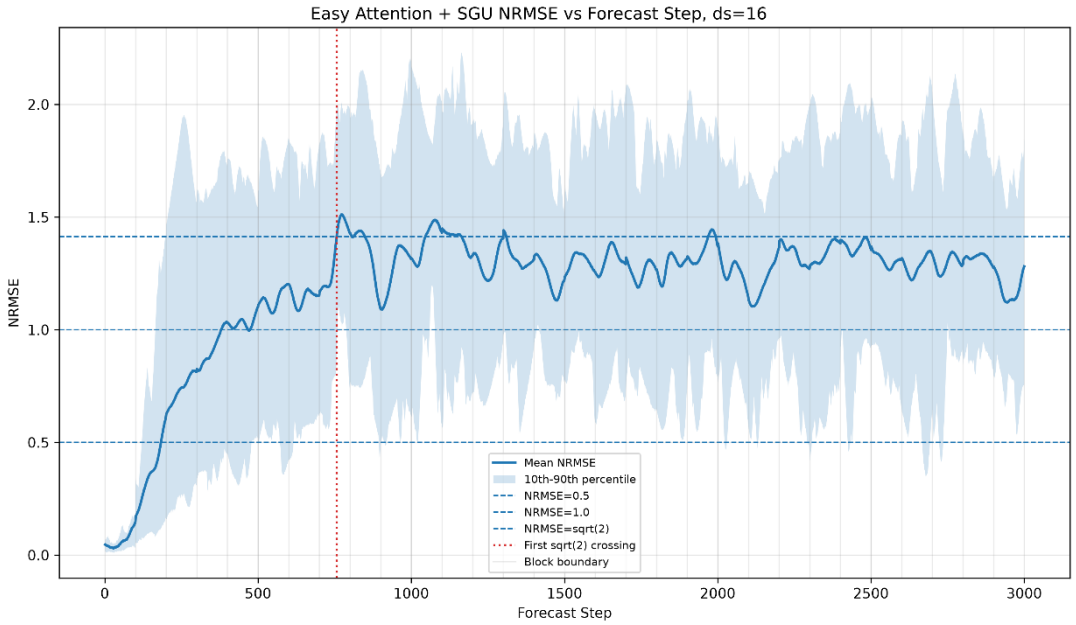


Figure 6.2. Easy Attention + SGU, ds = 16. The same 20-trajectory evaluation and fixed training-statistic normalization.

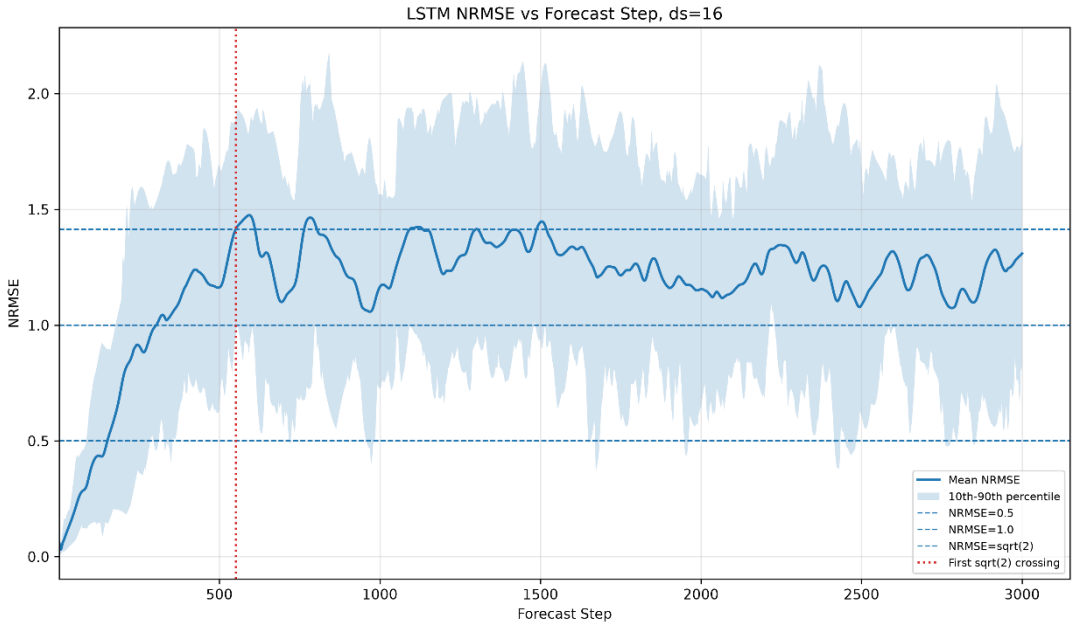


Figure 6.3. LSTM, ds = 16. The recurrent model is evaluated in closed loop, one predicted state at a time.

At ds = 16, the mean per-trajectory VPT at threshold 1 is 1.465 Lyapunov times for Easy Attention + SGU, 1.285 for Self Attention and 1.098 for LSTM. At the stricter 0.5 threshold the respective values are 0.975, 0.909 and 0.670 Lyapunov times. The combination model has the longest measured first-crossing time in this group, but its late-time curve still rises to order-one error. Its apparent advantage cannot be assigned to SGU alone because there is no otherwise matched Easy-Attention-only ablation in these archived final runs. All three mean curves fluctuate near the $\sqrt{2}$ reference at late times rather than proving a universal convergence to it; their 10th–90th percentile bands show substantial trajectory-to-trajectory spread.

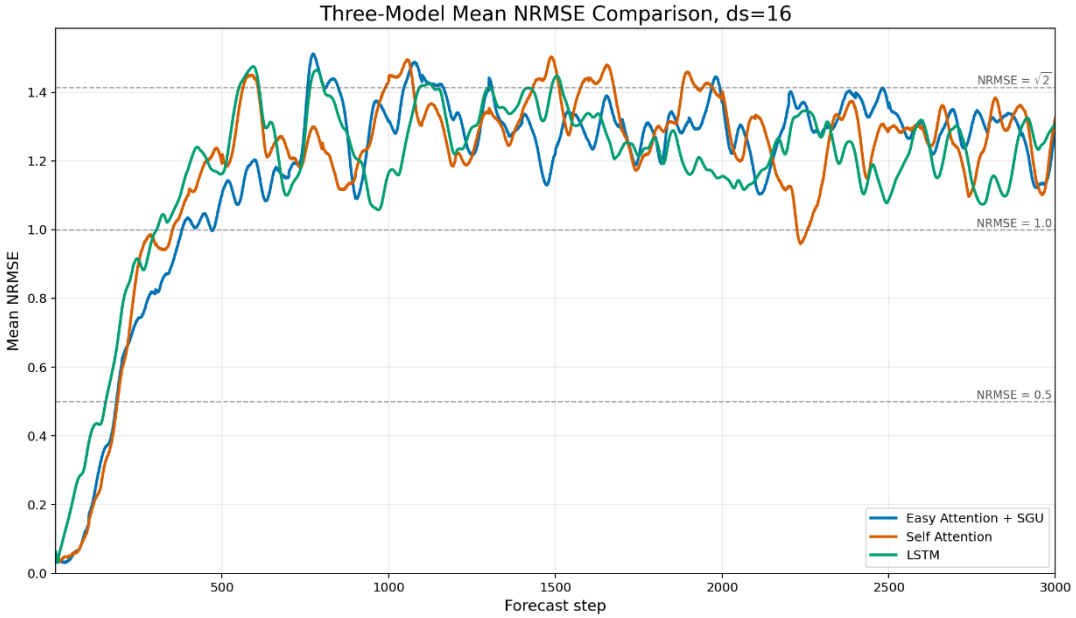


Figure 6.4. Three-model comparison at ds = 16. The two marked reference levels are NRMSE = 1 and $\sqrt{2}$; VPT is computed from individual curves, not this averaged overlay.

6.4 Results at ds = 32: two observed channels

Reducing the observed state to two channels makes the forecast less constrained by spatial information. The plots below should be read alongside their wide trajectory bands: one mean curve is not a guarantee about every initial condition. The ds change also changes the LSTM warm-up length from 128 to 256, so the difference between the LSTM groups is not a pure spatial-sampling ablation.

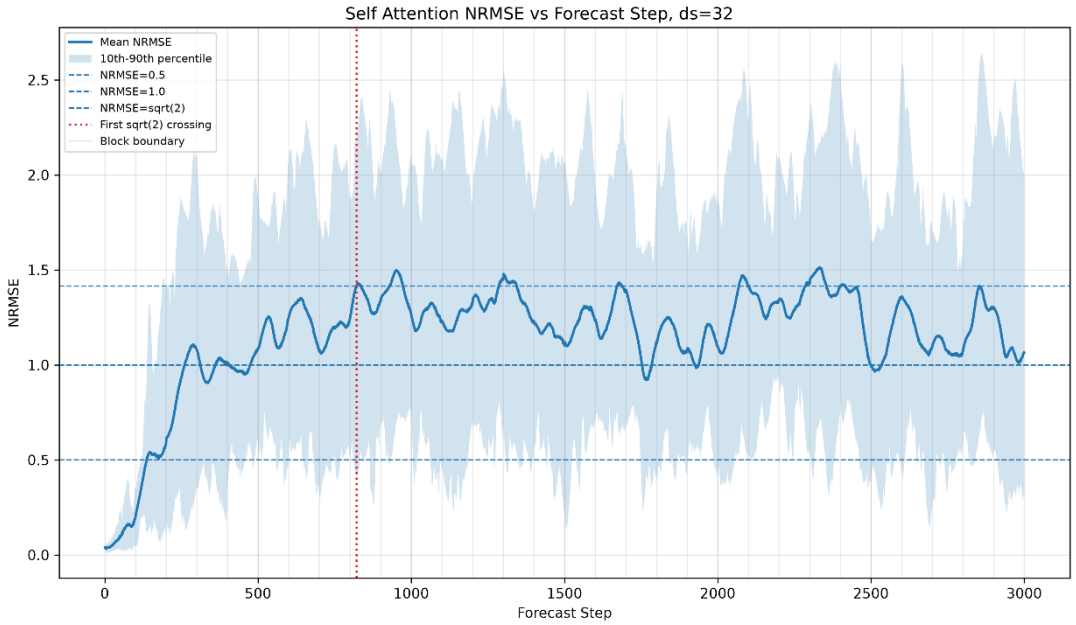


Figure 6.5. Self Attention, ds = 32. Twenty held-out trajectories; the mean reaches NRMSE = 1 earlier than at ds = 16.

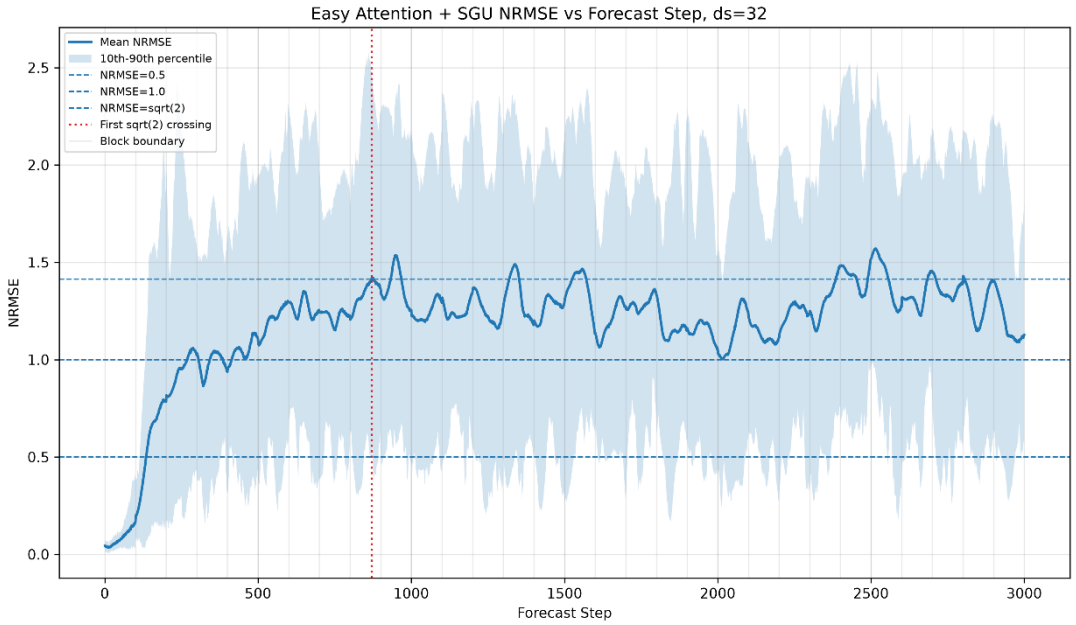


Figure 6.6. Easy Attention + SGU, ds = 32. The late-time mean can exceed the $\sqrt{2}$ reference, rather than settling on it.

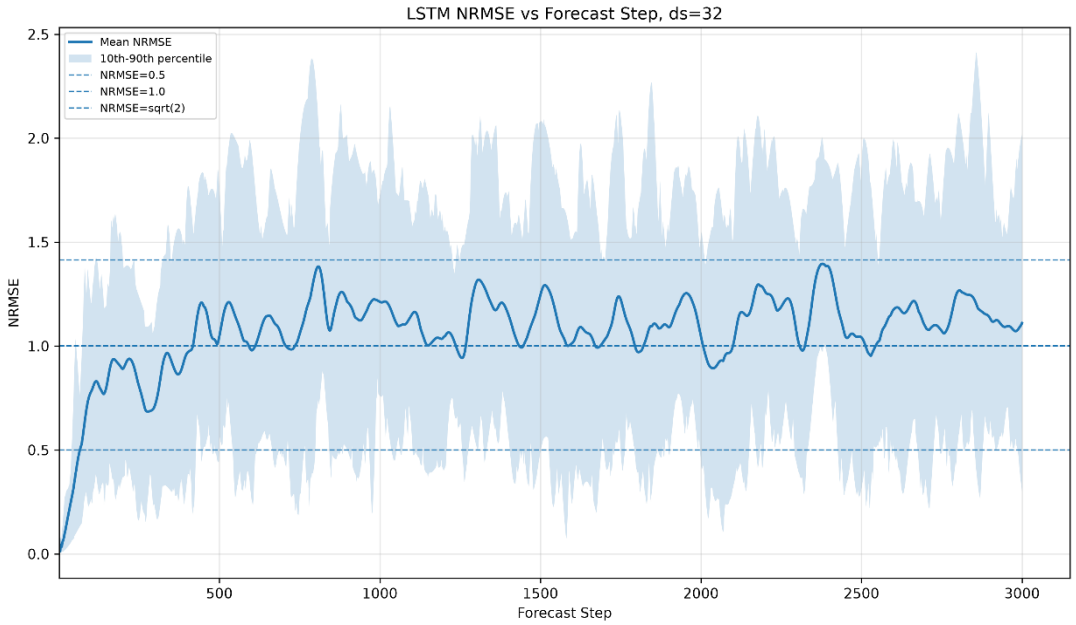


Figure 6.7. LSTM, ds = 32. The mean does not reach $\sqrt{2}$ in 3000 steps, but passes NRMSE = 1 and remains at large error.

At threshold 1, mean per-trajectory VPT is 1.227 Lyapunov times for Self Attention, 1.083 for Easy Attention + SGU and 0.681 for LSTM. At threshold 0.5 the values are 0.747, 0.780 and 0.436 Lyapunov times. Thus the two attention variants exchange order when the criterion changes: Easy Attention + SGU is slightly ahead at the stricter threshold, while Self Attention lasts longer to NRMSE = 1. For the ds = 32 LSTM, the mean first crossing occurs at 101.35 forecast steps for threshold 0.5 and 158.3 steps for threshold 1; multiplying by 0.0043 gives the stated VPTs. These values record only a short initial interval below each threshold. They say nothing about accuracy after the first crossing, and the 3000-step autonomous forecast fails. Its mean curve first reaches NRMSE = 1 at step 418, equivalent to 1.797 Lyapunov times, while the mean of individual first-crossing times is only 0.681 Lyapunov times. These are different statistics.

The ds = 32 LSTM mean peaks at about 1.396 and ends near 1.11, so it never reaches $\sqrt{2}$ during this finite horizon. That does not make it a 3000-step valid forecast: all 20 individual trajectories cross $\sqrt{2}$, even though their pointwise mean does not. The repeated excursions above NRMSE = 1 indicate loss of agreement with the target trajectory; remaining below $\sqrt{2}$ on average cannot rescue the long forecast. A mean near one could be compatible with biased or low-variation output, but this curve alone does not establish the mechanism. The two attention means also fluctuate near $\sqrt{2}$, and some late values exceed it. Neither a stable plateau nor universal convergence is supported by these curves.

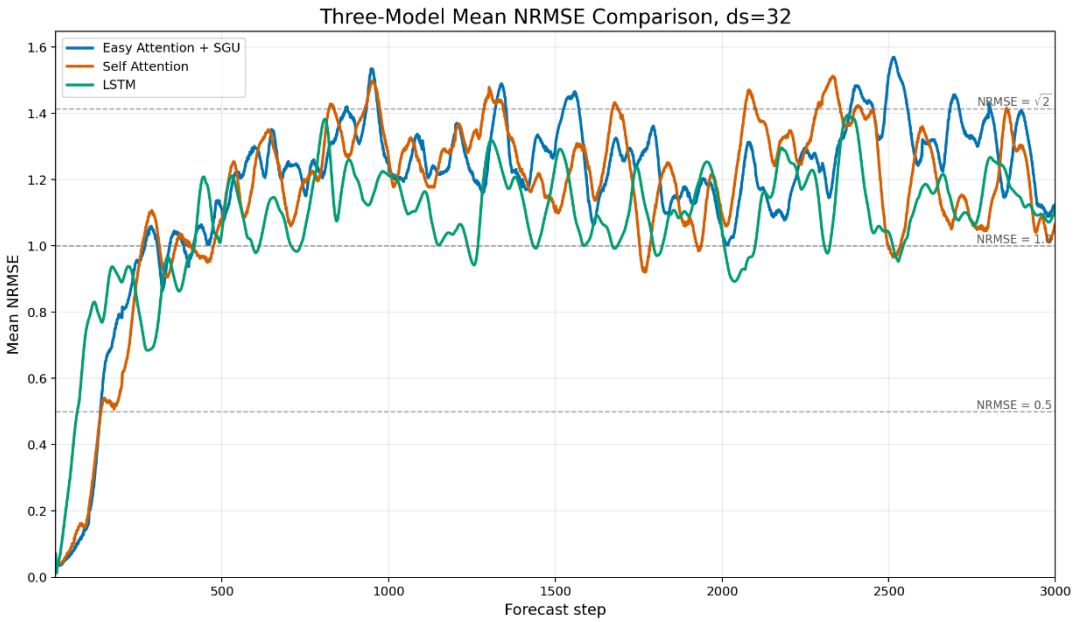


Figure 6.8. Three-model comparison at ds = 32. The vertical spread and late oscillations matter as well as the first threshold crossing.

6.5 First-crossing times in Lyapunov units

Figure 6.9 places all six runs on a common VPT scale. Each bar starts from a first crossing in an individual validation trajectory, is converted by 0.043 × 0.1 Lyapunov times per forecast step, and is then averaged over 20 trajectories. The NRMSE panels above retain forecast steps on their horizontal axes. The lower VPT from ds = 16 to ds = 32 is visible for all three pipelines under the threshold-1 criterion, but the differences are descriptive results from one trained checkpoint per setting, not a claim of statistical significance across independent training runs.

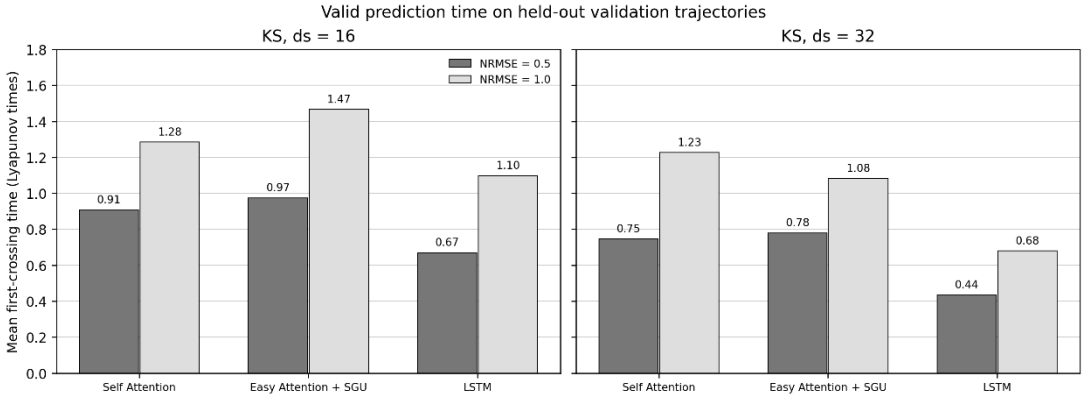


Figure 6.9. Mean first-crossing VPT in Lyapunov times at NRMSE thresholds 0.5 and 1 for six KS runs (20 held-out validation trajectories each). The reference exponent 0.043 is taken from the evaluation configuration, not measured here; √2 is not a VPT threshold.

6.6 Earlier Easy Attention runs at four other spatial resolutions

During the earlier exploratory stage I also saved Easy Attention curves at ds = 1, 2, 4 and 8. Figure 6.10 redraws only those four NRMSE series, preserving the forecast lead from 1 to 3000 and adding references at 1 and $\sqrt{2}$. These runs are historical context, not additional points in the six-run final comparison. The earlier prediction code selects 100 trajectories from the same dataset used for training; that set is not an independent test or a trajectory-isolated validation set. Its NRMSE divides the spatial mean-square error by the truth-field spatial variance at each predicted time, whereas the final runs use each channel's fixed training-set standard deviation. I therefore do not compare the numerical heights or first-crossing times of Figure 6.10 with Figures 6.1–6.9.

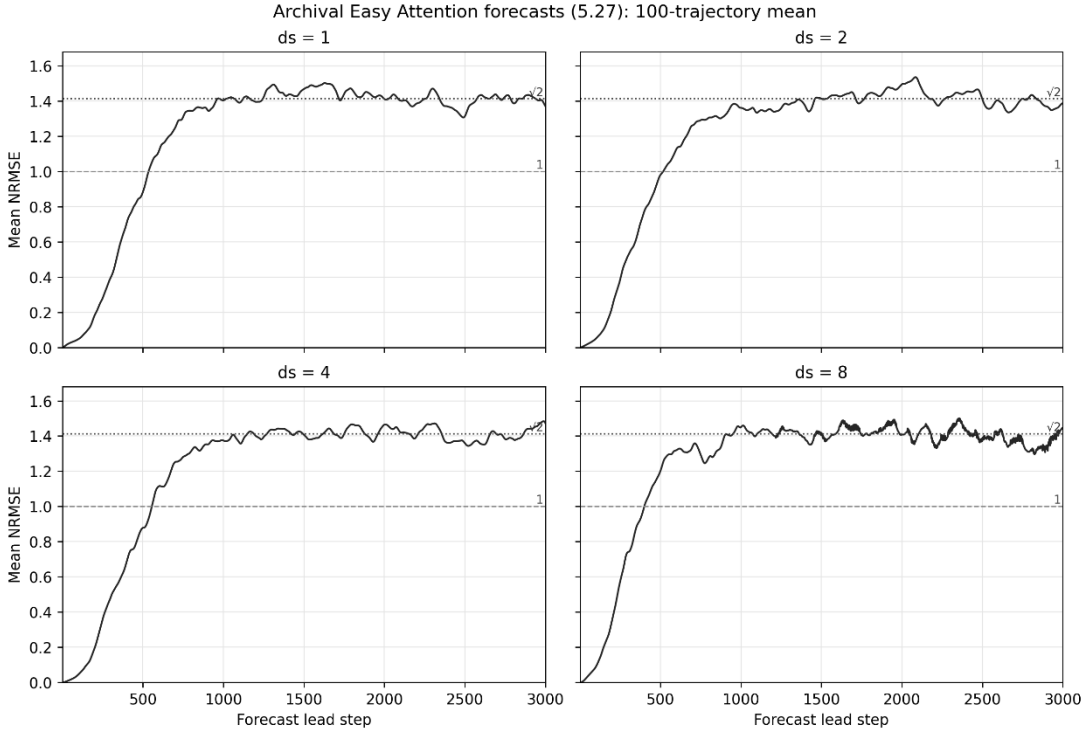


Figure 6.10. Historical Easy Attention mean NRMSE curves at ds = 1, 2, 4 and 8, redrawn from earlier exploratory results. These curves use the earlier metric and overlapping evaluation data; the horizontal lines at 1 and $\sqrt{2}$ are visual references only.

6.7 Curriculum loss and limits of the comparison

The revision fixed two clear methodological weaknesses of my earlier code: overlap between training and validation windows, and the mismatch between a three-step training target and one-step use at inference. It also makes the error scale fixed by training data rather than by each future true state. These changes make the evaluation more interpretable. They do not remove the normal sensitivity of a chaotic trajectory to accumulated small errors, and they do not prove that the architecture alone caused every difference between model curves.

The final LSTM code uses a closed-loop curriculum that the earlier prototype did not have. Its training rollout horizon H is 10 through epoch 20, 20 through epoch 50, 50 through epoch 100, and 100 thereafter. Figures 6.11 and 6.12 show the archived training and validation MSE at the current H for ds = 16 and 32. At each marked change the task becomes longer: the upward step in loss at epochs 20, 50 or 100 is therefore a change in the quantity measured, not by itself evidence that the weights became worse. Within each fixed-H stage, both curves generally decline. Absolute losses across stages measure different rollout lengths and cannot directly establish convergence or deterioration.

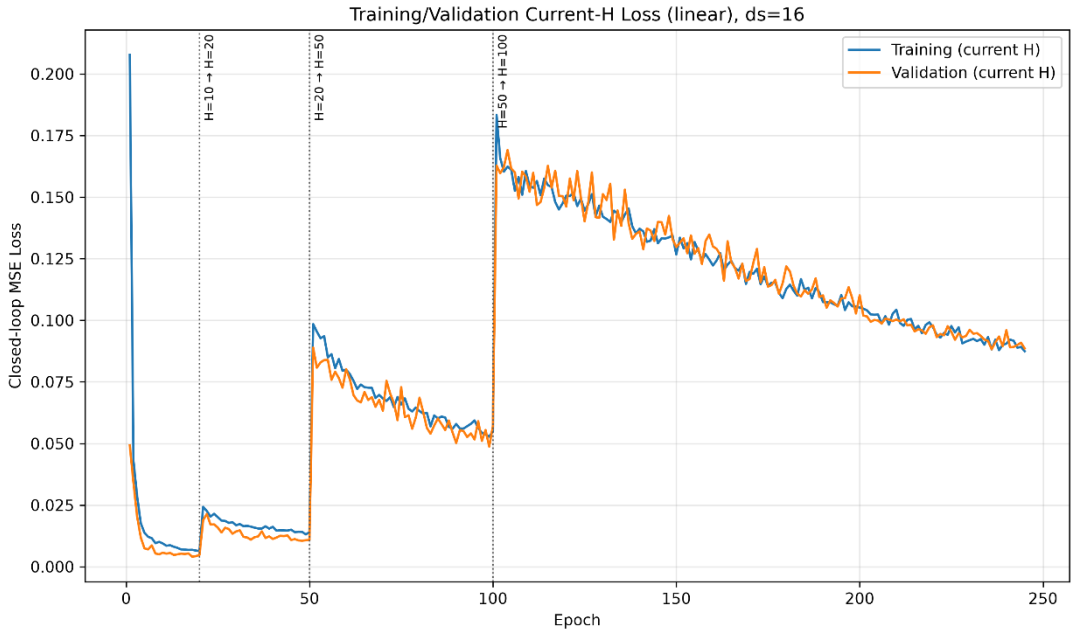


Figure 6.11. LSTM, ds = 16: current-horizon training and validation loss under the 10, 20, 50, 100-step curriculum. Values on opposite sides of a horizon change measure different tasks.

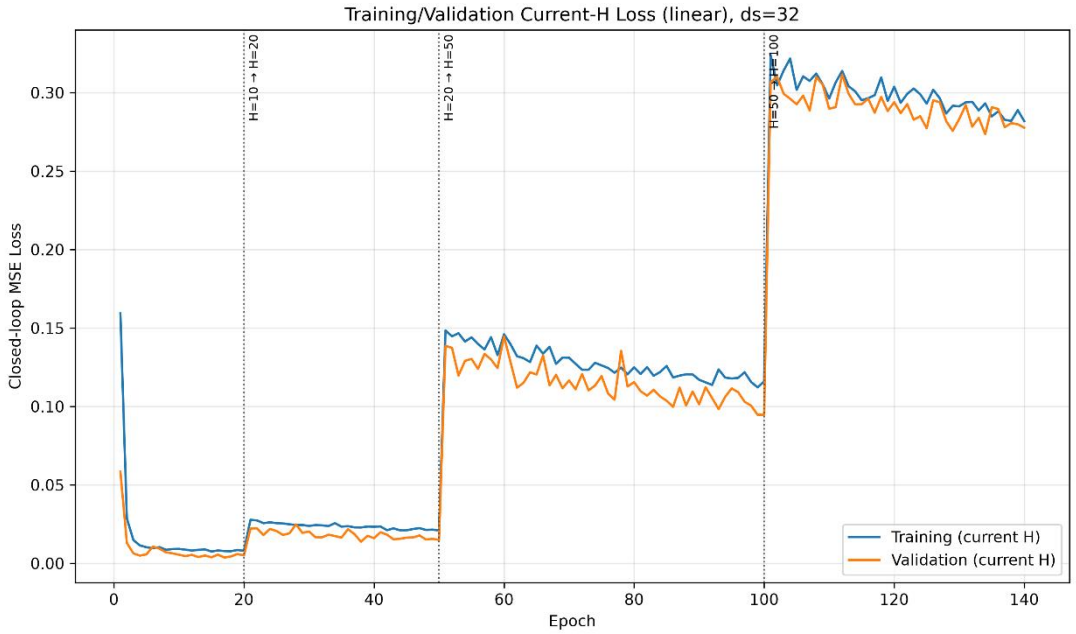


Figure 6.12. LSTM, ds = 32: current-horizon training and validation loss. The increases at epochs 50 and 100 coincide with a longer rollout target; loss then decreases within those stages.

The ds = 32 validation loss improves after the final horizon increase, but even a 100-step training diagnostic cannot establish a reliable 3000-step autonomous forecast. Its long-rollout NRMSE and individual trajectory crossings in Section 6.4 provide the direct evidence of failure at that scale. The early 5.27 Easy Attention runs in Figure 6.10 were not trained with this LSTM curriculum; no curriculum-loss claim is made for them.

Checkpoint selection itself uses held-out rollout behavior, and the same 20 validation trajectories are plotted here. Thus these results are a comparison on held-out validation trajectories, not an independent final test. The runs also differ in LSTM history length and training regime. A future experiment should fix the data, training budget and history length across the ablations, repeat training with several seeds, and reserve a separate trajectory set that is not touched during checkpoint selection.

With four observed KS channels, Easy Attention + SGU has the longest measured VPT at threshold 1 among these final runs. With only two channels, Self Attention has the longest VPT at that threshold. These VPT values quantify short initial intervals before first threshold crossing; none of the six curves supports sustained trajectory accuracy through the 3000-step horizon. In particular, the ds = 32 LSTM cannot make a reliable long autonomous prediction despite its finite early VPT; the fact that its mean NRMSE does not reach $\sqrt{2}$ is not evidence to the contrary. The late-time fluctuations and cases above $\sqrt{2}$ should be reported as loss of trajectory agreement, not recast as successful convergence.

References

[1] A. Vaswani *et al.*, “Attention Is All You Need,” *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.

[2] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” *International Conference on Learning Representations (ICLR)*, 2015.

[3] M. Sanchis-Agudo, Y. Wang, R. Arnau, L. Guastoni, J. Lim, K. Duraisamy, and R. Vinuesa, “Easy attention: A simple attention mechanism for temporal predictions with transformers,” arXiv:2308.12874, 2023 (revised 2025).

[4] A. Solera-Rico, C. Sanmiguel Vila, M. A. Gómez, Y. Wang, A. Almashjary, S. T. M. Dawson, and R. Vinuesa, “β-Variational autoencoders and transformers for reduced-order modelling of fluid flows,” Nature Communications, vol. 15, article 1361, 2024. doi:10.1038/s41467-024-45578-4.

[5] P. R. Vlachas, J. Pathak, B. R. Hunt, T. P. Sapsis, M. Girvan, E. Ott, and P. Koumoutsakos, “Backpropagation algorithms and reservoir computing in recurrent neural networks for the forecasting of complex spatiotemporal dynamics,” Neural Networks, vol. 126, pp. 191–217, 2020. doi:10.1016/j.neunet.2020.02.016.

Appendix A. Settings of the six revised KS runs

All rows use the 64-point periodic KS field on a domain of length 22, 100 trajectories split 80/20 by trajectory identity (seed 42), observation of every ds-th spatial grid point, saved interval 0.1, stride-one training windows, 3000-step autonomous evaluation, and the same 20 held-out validation trajectories. D is the number of observed channels. The output column states how the trained model is actually used during inference.

Model	ds	D	History	Output	Best epoch
Self Attention	16	4	128	100/block	350
Easy Attn. + SGU	16	4	128	100/block	310
LSTM	16	4	128	1/step	245
Self Attention	32	2	128	100/block	390
Easy Attn. + SGU	32	2	128	100/block	480
LSTM	32	2	256	1/step	140

The attention models have hidden dimension 256, eight heads, three layers and feed-forward width 1024. They use batch 32, AdamW, learning rate 0.0001, weight decay 0.0001 and cosine scheduling. LSTM uses hidden width 256, three layers, batch 64 and closed-loop curriculum training. Its scheduled horizons are 10, 20, 50 and 100, with learning rates 0.0005, 0.0003, 0.0002 and 0.0001. The table reports the saved epochs of the selected best-rollout checkpoints, not the last training epoch.

The metric uses per-channel training means and population standard deviations (ddof = 0), with a numerical standard-deviation lower bound of 10^{-8}. Reported VPTs are first-crossing times at NRMSE 0.5 and 1, expressed in Lyapunov times by multiplying each crossing step by 0.043×0.1. The six-run NRMSE figures were computed from saved per-trajectory predictions and errors. The historical ds = 1, 2, 4 and 8 curves come from an earlier exploratory stage and must be interpreted separately. Some preliminary configuration defaults, including the requested evaluation count, do not describe the completed run; the selected checkpoints and result summaries provide the settings reported here.